



Build Basirah From Zero

A Complete Beginner's Handbook for Building an Urdu-First,
On-Device Diabetic Retinopathy Screening App in 7 Days

ALIBABA CLOUD AI HACKATHON PAKISTAN 2026
Bano Qabil × Alibaba Cloud

A screening-aid prototype — not a certified medical device.
Written for readers with zero prior coding experience.

Table of Contents

How To Use This Handbook	4
---------------------------------	----------

PART A — FOUNDATIONS	4
-----------------------------	----------

A1. What Is Basirah?	5
A2. What Problem It Solves	5
A3. What The Finished App Actually Does	5
A4. A Few Terms You'll Need Right Away	5
A5. Our Technology Stack, and Why Each Piece Was Chosen	6
A6. How All The Pieces Talk To Each Other	7
A7. Final Architecture Overview	8
A8. What You Need Installed Before Starting	8
A9. Accounts, API Keys, and Credentials — Exactly Where They Come From and Where They Go	9
A10. The Project Folder & File Map (Preview)	10

PART B — THE 7-DAY BUILD	11
---------------------------------	-----------

DAY 1 — Accounts, Tools, and the Training Dataset	11
DAY 2 — Teaching the AI Model to Recognize Diabetic Retinopathy	17
DAY 3 — Improving the Model and Exporting It for the Phone	21
DAY 4 — Building the Flutter App and Wiring Up the AI Model	24
DAY 5 — Quality Checks, Real Results, and Safety Messaging	31
DAY 6 — Urdu Translation and Right-to-Left Layout	39
DAY 7 — Testing, Polish, and Demo Preparation	47

PART C — EVERYTHING ELSE YOU NEED	49
--	-----------

C1. Final Project Structure Explained	49
C2. The Complete AI / Image Pipeline, Stage by Stage	50
C3. Hackathon Optimization — What To Build, In Priority Order	51

C4. Testing — Complete Beginner-Friendly Checklist	52
C5. Deployment — Explained From Zero	53
C6. Demo Preparation — How To Present Basirah To Judges	54
C7. Troubleshooting — Master Reference	55
C8. Glossary	57

How To Use This Handbook

You are a complete beginner. This handbook assumes you have never written code, never used Git, never trained a machine learning model, and never built a mobile app. Every technical term is explained the first time it appears, in plain language, before it is used.

How the handbook is organized:

- **Part A — Foundations.** What Basirah is, why it exists, what technology we're using and why, and everything you need installed before Day 1 starts.
- **Part B — The 7-Day Build.** The actual construction, one day at a time. Every step tells you exactly which file to open, what to type or paste, and what you should see happen.
- **Part C — Everything Else You Need.** Project structure reference, the AI pipeline explained end-to-end, hackathon priorities, testing, "deployment" (which, for this project, is simpler than you'd expect), demo preparation, troubleshooting, and a glossary.

Conventions used throughout this book:

TIP

Tip boxes look like this. They contain optional advice that makes your life easier but isn't required.

WARNING

Warning boxes look like this. They flag a step where beginners commonly make a mistake, or something that could cost you real time if you skip it.

CHECKPOINT

Checkpoint boxes look like this. They appear after a milestone and tell you exactly what "working" looks like, and what to do if it doesn't.

Code you need to type or paste appears in a shaded box like this:

BASH

```
this is a command you would type into a terminal
```

The little label above a code box (`bash`, `dart`, `python`, `powershell`) tells you *where* that code goes — into a terminal, or into a specific kind of file. We'll always tell you explicitly which file too.

One honest note before we start: this handbook builds a real, working, on-device AI screening tool — not a toy. But it is a **hackathon MVP screening prototype**, not a certified medical device. Every part of the build — including the wording shown to end users — is designed to make that distinction clear. You'll see why this matters in Part A.

PART A — FOUNDATIONS

A1. What Is Basirah?

Basirah (بصیرت — Urdu for "insight" or "vision") is a mobile app that looks at a photograph of the back of a person's eye (a **retinal fundus photo**) and gives a preliminary screening opinion about whether it shows signs of **diabetic retinopathy** — an eye disease caused by diabetes that is one of the leading causes of preventable blindness worldwide.

Basirah is built specifically for Pakistan, in Urdu as well as English, and is designed to work entirely offline, on ordinary low-cost Android phones.

A2. What Problem It Solves

Diabetes is extremely common in Pakistan, and diabetic retinopathy has no symptoms in its early, most treatable stages — by the time a person notices vision problems, damage may already be advanced. Catching it early requires a routine eye photo and grading, but Pakistan has a severe shortage of ophthalmologists (eye doctors) and eye-care infrastructure, especially outside big cities. Research on Pakistan's eye-care system confirms there is no population-wide diabetic retinopathy screening program in most facilities, and that cost, distance, and lack of awareness keep many people from ever getting screened.

Basirah doesn't replace an eye doctor. It gives ordinary people (and community health workers) a way to get a **preliminary signal** — "this looks fine for now, but keep up routine checks" or "this shows signs that should be checked by a specialist soon" — using nothing but a phone and a fundus camera photo, with no internet connection required and no image ever leaving the device.

A3. What The Finished App Actually Does

Walking through it as a user would experience it:

- Open the Basirah app on an Android phone.
- Choose your language — English or Urdu (with the whole app switching direction: Urdu reads right-to-left).
- Take or import a photo of a retinal fundus image (in a real deployment this photo usually comes from a specialized fundus camera; for our MVP and demo, we'll use sample fundus images from a public research dataset).
- The app checks whether the photo is usable (not blurry, not too dark/bright, actually looks like a fundus photo). If it isn't, the app asks you to retake it — before it ever tries to analyze a bad photo.
- The app processes the image and runs it through an on-device AI model.
- Within about a second, the app shows a plain-language result: **"No urgent referral"** or **"Please see an eye-care professional."** The result screen is explicit that this is a *screening aid*, not a diagnosis, and always recommends professional follow-up.
- Nothing about the photo or the result is ever sent anywhere. It all happens on the phone.

A4. A Few Terms You'll Need Right Away

We'll build up a full glossary at the end (Part C), but here are the handful of terms you'll see constantly from page one, explained simply:

- **App / Frontend** — the part of the software the user actually sees and touches: buttons, screens, images on the phone. In Basirah, this is the entire app — there is no separate website.

- **Backend** — normally, a separate program running on a server somewhere that the app talks to over the internet (for example, to check a password or fetch data). **Basirah has no backend.** Everything happens on the phone itself. We'll explain why this is a deliberate, good choice for this project in A7.
- **AI Model** — think of this as a very elaborate lookup table that has "learned" from thousands of example photos to recognize a pattern (in our case: signs of diabetic retinopathy) and estimate how confident it is. It is not a person, it doesn't "understand" anything the way a doctor does — it recognizes visual patterns statistically.
- **Inference** — the act of actually *running* a trained AI model on a new photo to get a result. "Training" happens once, ahead of time, on a computer. "Inference" happens every time a user analyzes a photo, and in our case it happens on the user's phone.
- **API (Application Programming Interface)** — think of this as a waiter between two software systems. One system asks for something (like "give me this user's data"), the waiter (the API) carries the request to the other system, and brings the response back. Basirah, because it has no backend, barely uses this idea at all — which is part of why it's beginner-friendly to build.

A5. Our Technology Stack, and Why Each Piece Was Chosen

Purpose	Technology	Plain-language explanation
The mobile app itself	Flutter (using a programming language called Dart)	Flutter is a toolkit made by Google for building one app that runs on both Android and iPhone from a single set of code. Think of it like a set of Lego bricks specifically for building app screens — buttons, text, images, layouts — that Google maintains and documents extremely well.
Running the AI model on the phone	TensorFlow Lite / LiteRT (via a Flutter package called <code>tflite_flutter</code>)	This is a lightweight version of Google's AI toolkit, specifically shrunk down to run efficiently on phones instead of powerful servers. It's the piece that actually does the "look at this photo and estimate diabetic retinopathy risk" work, entirely offline.
Training the AI model (done once, before the app exists)	Python with TensorFlow/Keras	Python is the most common programming language for machine learning, and Keras is a beginner-friendly way to build and train AI models with it. We use this on a cloud computer (not your phone) to <i>teach</i> the model, once, using thousands of example photos. The result is a small file that the phone can then run.
Where the training happens	Alibaba Cloud PAI-DSW (primary — this hackathon's sponsor platform), with Google Colab as a zero-signup fallback	Training an AI model needs a powerful graphics card (GPU) that your everyday laptop probably doesn't have fast enough. Both of these are "rent a powerful computer in the cloud, for free or via hackathon credits, through your web browser" services. We explain both, fully, in Day 1.

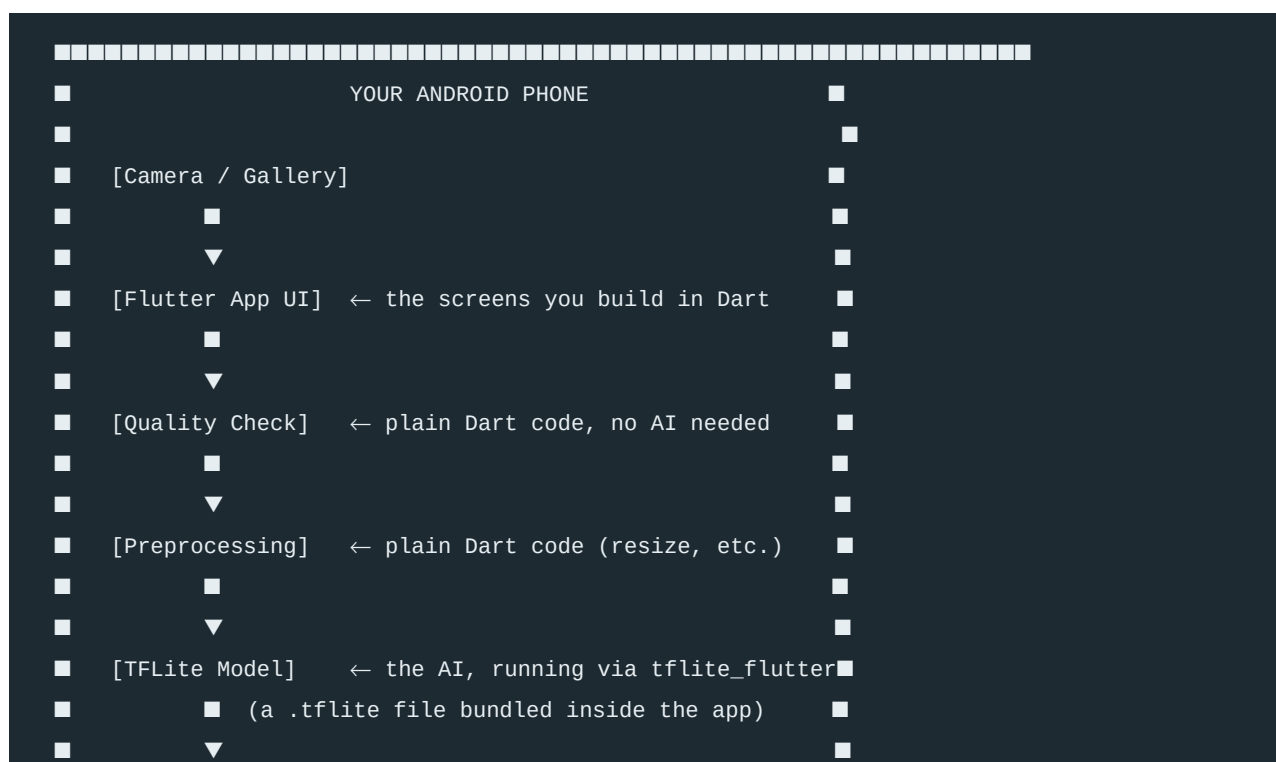
Purpose	Technology	Plain-language explanation
The example photos the AI learns from	APTOS 2019 dataset (from Kaggle, a public data-science website)	A public, hospital-sourced collection of about 3,700 real retinal photos, each already labeled by doctors with a diabetic retinopathy severity grade. This is what "teaches" our model.
Two languages, right-to-left support	Flutter's built-in localization system (<code>flutter_localizations</code> , <code>intl</code>)	Flutter has first-class, official support for translating an app into multiple languages and automatically flipping the whole layout for right-to-left languages like Urdu. This isn't a workaround — it's a fully supported, documented feature.

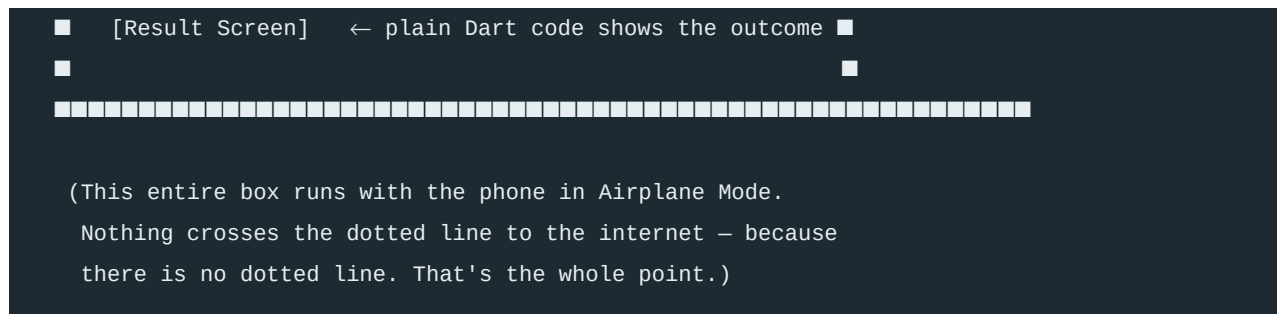
Why no backend, no database, no cloud server for the finished app? Three good reasons, not laziness:

- **Privacy.** A retinal photo is sensitive medical data. If it never leaves the phone, there's nothing to leak, hack, or mishandle.
- **Connectivity.** Much of Pakistan has unreliable or expensive internet. An app that requires the internet to work is an app that doesn't work reliably for its actual target users.
- **You're a beginner with 7 days.** Building and hosting a backend server correctly (with security, uptime, and error handling) is a significant extra project on its own. Removing it removes an entire category of things that can go wrong — while still building something genuinely more useful for the people it's meant for.

A6. How All The Pieces Talk To Each Other

Because there's no backend, this is simpler than almost any other app you'll ever build. Everything below happens **inside the one Flutter app, on one phone**:





The **only** place the internet is involved at all is *before* the app exists: while you, the developer, are training the AI model on a cloud computer (Alibaba Cloud or Colab) during Days 1–3. Once the trained model is exported as a small file and placed inside the app, the internet is no longer needed for the app to work.

A7. Final Architecture Overview

- **One Flutter project** ([app/](#)), producing one installable Android app.
- **No backend server. No database. No user accounts. No login.**
- **One bundled AI model file** (a few megabytes) shipped inside the app.
- **All image processing and AI inference happen on-device**, using the phone's own processor.
- **Separately, a one-time Python training pipeline** ([m1/](#)) — this is *not* part of the final app; it's the "kitchen" where we cook the AI model once, before baking it into the app.

This is deliberately the simplest architecture that can honestly deliver everything the project needs. There is nothing to "cut" here to simplify further — the complexity we do have (a real trained AI model, running on a phone, in two languages) is the actual point of the project.

A8. What You Need Installed Before Starting

You'll install these on your own Windows computer. We'll walk through each one on the day it's first needed, but here's the full list up front so you know what's coming:

Software	What it's for	Needed by
Git for Windows	Lets your computer track changes to your project files and talk to GitHub (explained in Day 1).	Day 1
A GitHub account	A free website that stores a backup copy of your code "in the cloud" and is the standard way developers share and back up projects.	Day 1
Python 3.10 or newer	The language we use for AI training scripts (only used on the cloud training machine, not usually on your own PC, but useful to have).	Day 2
A Kaggle account	Free data-science website where we'll download our training photos from.	Day 1
A Google account	Needed for Google Colab (our training-compute fallback) and generally useful.	Day 1

Software	What it's for	Needed by
An Alibaba Cloud account	This hackathon's sponsor platform — gives you free/credited access to a powerful cloud computer for AI training.	Day 1 (optional path)
Flutter SDK	The toolkit that builds our mobile app.	Day 4
Android Studio	Provides the Android tools Flutter needs, plus an Android emulator (a virtual phone on your computer) and USB drivers for a real phone.	Day 4
Visual Studio Code (VS Code)	A free, beginner-friendly code editor — where you'll actually write and paste code.	Day 1
An Android phone (recommended)	For testing the real app on real hardware. An emulator works too, but a real phone is faster and more reliable for beginners, and lets you use a real camera.	Day 4 onward

TIP

Don't install everything today. Each day of Part B tells you exactly what to install *that day*, right before you need it, so you're never staring at a wall of unfamiliar software with nothing to do with it yet.

A9. Accounts, API Keys, and Credentials — Exactly Where They Come From and Where They Go

A **credential** (or **API key**, or **token**) is just a secret piece of text that proves "this request is really coming from you" — like a password, but generated by the computer instead of chosen by you. Here is every credential this project needs:

Credential	Where you get it	Where it goes	Never do this
Kaggle API token (<code>kaggle.json</code>)	Kaggle.com → click your profile picture → Account → scroll to API section → Create New Token . This downloads a file called <code>kaggle.json</code> .	On your training machine (Colab or Alibaba PAI-DSW), placed in a special hidden folder — exact steps in Day 1.	Never upload <code>kaggle.json</code> to GitHub or paste its contents into any chat, code file, or screenshot you share publicly.
Alibaba Cloud account credentials (login email/password, and an AccessKey if you use the command line)	Created directly at alibabacloud.com when you sign up.	You log into the Alibaba Cloud console with them in your browser — you generally won't type these into any code file for this project.	Never commit an AccessKey ID/Secret into any file in your project folder.
Google account login	Your existing Google account, or a new free one.	Used only to log into colab.research.google.com in your browser. Nothing to place in a file.	—

Credential	Where you get it	Where it goes	Never do this
GitHub account login	Free signup at github.com.	Used to log in via Git on your computer (Day 1 shows exactly how).	—

That's the entire list. Notice there is no "app secret key," no database password, no payment API key — because Basirah's *finished app* has no backend and needs none of those. The only real secret in this whole project is your Kaggle token, and it's only ever used on the training machine, never inside the app itself.

A10. The Project Folder & File Map (Preview)

Here is the complete folder structure we will build over the 7 days. Don't worry about understanding every line yet — we'll create each piece exactly when we need it, and Part C explains every folder in detail.

```

basirah/
├── plan.md                ← your living project plan/checklist (open this daily)
├── docs/                 ← written project documents (dataset notes, safety copy,
    etc.)
├── ml/                   ← the AI training project (Python, runs in the cloud)
│   ├── data/             ← downloaded training photos (never uploaded to GitHub)
│   ├── src/
│   │   ├── preprocess.py
│   │   ├── train.py
│   │   ├── evaluate.py
│   │   └── export_tflite.py
│   └── models/           ← the trained .tflite file ends up here
├── app/                  ← the Flutter mobile app
│   ├── lib/
│   │   ├── main.dart
│   │   ├── pipeline/
│   │   │   ├── quality_gate.dart
│   │   │   ├── preprocessing.dart
│   │   │   ├── inference_service.dart
│   │   │   └── result_mapper.dart
│   │   ├── screens/
│   │   │   ├── home_screen.dart
│   │   │   └── result_screen.dart
│   │   ├── l10n/
│   │   │   ├── app_en.arb
│   │   │   └── app_ur.arb
│   │   ├── assets/
│   │   │   ├── models/basirah_model.tflite
│   │   │   └── fonts/NotoNastaliqUrdu-Regular.ttf
│   │   └── pubspec.yaml

```

You now have everything you need to understand *what* we're building and *why*. Part B builds it, one day at a time.

PART B — THE 7-DAY BUILD

DAY 1 — Accounts, Tools, and the Training Dataset

Day Objective

By the end of today: your computer has every tool installed that today's work needs, you have every account created, your project folder exists on GitHub, and the training photos (APTOS 2019) are downloaded and converted into the simple "Refer / No urgent referral" labels our model will learn from.

Concepts You Need Today

- **Terminal / Command line** — a text-only way to give your computer instructions, instead of clicking icons. You type a command, press Enter, and read the text that comes back. On Windows we'll use **PowerShell** (search "PowerShell" in your Start menu).
- **Git & GitHub** — Git is a program that tracks every change you make to your project's files, like an infinite "undo history" with labeled save points. GitHub is a website that stores a copy of that history online, so your work is backed up and shareable. A **repository** ("repo") is just the name for one project's folder-plus-history.
- **Environment variable** — a named piece of information your computer's programs can read, kept outside your actual code files. We don't need to create any today, but you'll see the term again in Day 6.
- **CSV file** — "Comma-Separated Values" — the simplest possible spreadsheet file format: plain text, one row per line, columns separated by commas. Our dataset's labels come as a CSV.

Setup — Install These Now

1. Install Git for Windows

Download from git-scm.com/download/win and run the installer. Accept all the default options — you don't need to change anything.

Verify it worked — open PowerShell and run:

```
POWERSHELL
git --version
```

You should see something like `git version 2.4x.x`. If you see `'git' is not recognized...`, close and reopen PowerShell (Git adds itself to your system PATH, which sometimes needs a fresh terminal window to take effect), then try again.

2. Create a GitHub account

Go to github.com and sign up (it's free). Remember your username — you'll use it constantly.

3. Tell Git who you are (one-time setup, run in PowerShell):

```
POWERSHELL
git config --global user.name "Your Name"
git config --global user.email "you@example.com"
```

Use the same email you used for GitHub.

4. Install Visual Studio Code

Download from code.visualstudio.com and install it with default options. This is the program you'll use to open, read, and edit every code file in this handbook. Whenever we say "open `some/file.dart`," we mean: open VS Code, open your project folder in it, and click that file in the sidebar.

5. Create your project folder and put it on GitHub

In PowerShell:

```
POWERSHELL
cd D:\
mkdir basirah
cd basirah
git init
```

`git init` turns this ordinary folder into a Git repository — it starts tracking changes here from now on.

Now go to github.com, click the + icon top-right → **New repository**. Name it `basirah`, leave it **empty** (don't check "Add a README"), and click **Create repository**. GitHub will show you a page with commands — copy the ones under "...or push an existing repository from the command line," which look like this (use YOUR username):

```
POWERSHELL
git remote add origin https://github.com/YOUR-USERNAME/basirah.git
git branch -M main
```

We won't push (upload) anything yet — there's nothing to save. We'll do our first real push at the end of today, once we have actual files.

TIP

What is "pushing"? Your Git history lives on your own computer first. "Pushing" copies your saved history up to GitHub, so it's backed up online. You'll do this at the end of every day.

6. Create a Kaggle account and get your API token

Go to kaggle.com and sign up. Once logged in:

- Click your profile picture (top-right) → **Settings**.
- Scroll to the **API** section.
- Click **Create New Token**.
- A file named `kaggle.json` downloads to your computer. **Keep this file safe — don't share it, don't upload it anywhere public.** It contains your personal Kaggle credentials.

We'll use this file in the next step, inside whichever training environment you choose.

Choosing Your Training Computer: Alibaba Cloud vs. Google Colab

Training the AI model needs a computer with a powerful graphics card (a **GPU**) — far more powerful than what image classification training needs on a regular laptop CPU. We rent one for free, in the browser,

using one of two options. **Try Option A first, but give yourself a hard 45–60 minute time limit** — if account setup is dragging on, switch to Option B immediately. Losing half a day to cloud account verification is the single easiest way to derail a one-week hackathon build.

Option A — Alibaba Cloud PAI-DSW (uses your hackathon credits)

- Go to alibabacloud.com and sign up for an international account. You'll be asked for basic info and, in some cases, payment/identity verification details for account security — this does **not** mean you'll be charged; hackathon participants use provided credits/quota.
- Once logged into the **Alibaba Cloud Console**, search for **"PAI"** (Platform for AI) in the top search bar and open it.
- Create a **default workspace** if prompted (accept the defaults).
- Open **DSW (Data Science Workshop)** from the PAI menu, and click **Create Instance**.
- Choose a GPU instance type (an option like `ecs.gn6v-c8g1.2xlarge` is a reasonable, commonly available choice) and a pre-built image that includes **TensorFlow** (look for an image name containing `tensorflow`).
- Once the instance status shows **Running**, click **Open** to launch the notebook interface (it looks like Jupyter Notebook — a page with code cells you can run one at a time in your browser).

WARNING

If at any point you hit a wall — waiting on identity verification, unclear billing prompts, no GPU quota available yet — **stop and switch to Option B**. You can always come back to Alibaba Cloud later if time allows; don't let it block Day 1.

Option B — Google Colab (the simple, zero-signup fallback)

- Go to colab.research.google.com and sign in with your Google account. That's it — no setup.
- Click **New Notebook**.
- Turn on the free GPU: menu **Runtime** → **Change runtime type** → **Hardware accelerator** → **GPU (T4)** → **Save**.

Either way, you now have a notebook: a page in your browser where you can type Python code into a **cell** and click the little  play button (or press Shift+Enter) to run just that cell. Output appears right below it. This is where all of Day 1–3's Python code will run.

Step-by-Step: Download and Prepare the Dataset

From here, the steps are **identical** whether you're in Alibaba Cloud PAI-DSW or Google Colab — both give you a notebook of Python code cells.

Step 1 — Upload your `kaggle.json`

In Colab: run this in a new cell, click , then click the "Choose Files" button that appears and select your downloaded `kaggle.json`:

PYTHON

```
from google.colab import files
files.upload() # select kaggle.json when prompted
```

In Alibaba Cloud PAI-DSW: use the notebook's file-upload button (usually a small upload icon in the file browser sidebar) to upload `kaggle.json` into the working directory.

Step 2 — Install the Kaggle tool and place your token correctly

Paste this into a new cell and run it:

PYTHON

```
!pip install -q kaggle
!mkdir -p ~/.kaggle
!cp kaggle.json ~/.kaggle/
!chmod 600 ~/.kaggle/kaggle.json
```

What this does, line by line: the `!` at the start means "run this as a terminal command, not Python." Line 1 installs the `kaggle` tool. Line 2 creates the hidden folder Kaggle's tool expects to find your token in. Line 3 copies your uploaded token there. Line 4 locks the file's permissions so only you can read it — Kaggle's tool actually refuses to run without this on shared systems.

Step 3 — Download the APTOS 2019 dataset

WARNING

You must first **join the competition** on Kaggle's website before the API is allowed to download it (this is a one-click Kaggle rule for this specific dataset, not something we can skip). Open kaggle.com/competitions/aptos2019-blindness-detection, log in, and click **"I Understand and Accept"** on the rules page. This confirms you agree to Kaggle's non-commercial/research-use terms for this data — which matches exactly what Basirah is: a research-stage hackathon prototype, not a commercial product.

PYTHON

```
!kaggle competitions download -c aptos2019-blindness-detection -f train.csv
!kaggle competitions download -c aptos2019-blindness-detection -f train_images.zip
!unzip -q train_images.zip -d train_images
```

This downloads the label spreadsheet (`train.csv`) and a zip file of ~3,662 fundus photos, then unzips the photos into a folder called `train_images`.

CHECKPOINT

Checkpoint 1 — Your training environment works.

Run `!ls train_images | head -5` in a new cell. You should see a short list of filenames ending in `.png`, like `000c1434d8d7.png`. If instead you see an error mentioning "403 Forbidden," go back and make sure you accepted the competition rules on Kaggle's website (Step 3's warning above) — this is the single most common Day 1 error, and re-running the download after accepting the rules fixes it every time.

Step 4 — Convert the 5-class labels into our simple binary label

Open `train.csv` and look at it — it has two columns: `id_code` (the photo's filename, without `.png`) and `diagnosis` (a number 0–4, where 0 = No DR, 1 = Mild, 2 = Moderate, 3 = Severe, 4 = Proliferative DR).

Basirah's result screen only ever shows two outcomes — **"No urgent referral"** or **"Refer"** — because that's a simpler, more honest, and more clinically standard way to communicate a screening result than a 5-level grade (this exact grouping, "moderate-or-worse is referable," is standard practice in real DR screening programs, not something we invented). Run this in a new cell:

PYTHON

```
import pandas as pd

df = pd.read_csv("train.csv")

# Grades 0-1 (No DR, Mild) -> 0 = "No urgent referral"
# Grades 2-4 (Moderate, Severe, Proliferative) -> 1 = "Refer"
df["refer_label"] = (df["diagnosis"] >= 2).astype(int)

print(df["refer_label"].value_counts())
df.to_csv("train_binary.csv", index=False)
```

What this code does: `pd.read_csv` loads the spreadsheet into a table-like object called a **DataFrame**. `df["diagnosis"] >= 2` checks every row and produces `True/False`; `.astype(int)` turns that into `1/0`. We save the result as a new column, print how many images fall into each class (so you can see the class balance with your own eyes), and save a new CSV file.

CHECKPOINT**Checkpoint 2 — Your dataset is ready.**

You should see printed output roughly like:

```
0 2779
```

```
1 883
```

(Exact numbers may vary slightly by APTOS version, but you should see noticeably more `0`s than `1`s — that's the class imbalance we planned for in Day 2's training step. If you instead get a `KeyError` mentioning `'diagnosis'`, open `train.csv` and check the column name matches exactly — Kaggle occasionally updates file headers.)

Step 5 — Save your work so far

In Colab, your notebook and downloaded files disappear when the session ends unless you save them. Mount your Google Drive and copy the important files there:

PYTHON

```
from google.drive import drive # if this import fails, use: from google.colab import drive
drive.mount('/content/drive')
!mkdir -p /content/drive/MyDrive/basirah/data
!cp train_binary.csv /content/drive/MyDrive/basirah/data/
!cp -r train_images /content/drive/MyDrive/basirah/data/
```

In Alibaba Cloud PAI-DSW, your DSW instance's storage persists between sessions automatically (that's one of its advantages over Colab) — no extra step needed, just don't delete the instance.

Step 6 — Create your local repo skeleton and push to GitHub

Back on your own computer, in VS Code's terminal (menu **Terminal** → **New Terminal**) inside your `D:\basirah` folder:

POWERSHELL

```
mkdir docs, ml, ml\data, ml\src, ml\models, app
New-Item -ItemType File .gitignore
```

Open the new `.gitignore` file in VS Code and paste this in:

```
# .gitignore — tells Git which files to NEVER track or upload
ml/data/
ml/models/*.h5
*.tflite
kaggle.json
.dart_tool/
build/
.env
```

TIP

What is `.gitignore`? It's a plain list of file/folder patterns that Git should ignore completely — never track, never upload. We list `ml/data/` because the dataset's license doesn't allow redistribution (Day 1's Kaggle rules acceptance was personal to your account, not something you're allowed to re-share by uploading it to your public GitHub repo). We list `kaggle.json` so your personal credentials can never accidentally get uploaded either.

Now save your first checkpoint to Git and GitHub:

POWERSHELL

```
git add .
git commit -m "Day 1: project skeleton and dataset preparation notes"
git push -u origin main
```

What these three commands do: `git add .` stages every changed file (marks it "ready to save"). `git commit -m "..."` actually saves that snapshot to your local history, with a short message describing what changed. `git push` uploads that snapshot to GitHub. You'll repeat this exact three-command pattern at the end of every day.

Troubleshooting Day 1

Problem	Fix
<code>git : The term 'git' is not recognized...</code>	Close and reopen PowerShell after installing Git. If it still fails, restart your computer — PATH changes sometimes need a full restart on Windows.
Kaggle download gives <code>403 - Forbidden</code>	You haven't accepted the competition rules on the Kaggle website yet. Visit the competition page while logged in and click "I Understand and Accept," then re-run the download cell.
Kaggle download gives <code>401 - Unauthorized</code>	Your <code>kaggle.json</code> wasn't copied into the right hidden folder, or is malformed. Re-run Step 2's four commands.

Problem	Fix
Colab session disconnects and you lose your files	This is normal — Colab's free tier isn't persistent. Always run Step 5's Google Drive save <i>before</i> taking a long break.
<code>git push</code> asks for a password and rejects it	GitHub no longer accepts your account password for this. When prompted, use a Personal Access Token instead (GitHub → Settings → Developer settings → Personal access tokens → Generate new token, paste it in place of your password when Git asks) — or simply let VS Code's Git integration open a browser sign-in window, which is usually easier for beginners.

DAY 2 — Teaching the AI Model to Recognize Diabetic Retinopathy

Day Objective

By the end of today, you will have trained a first working version of the AI model and printed real accuracy numbers — proving the entire "photo in → prediction out" learning pipeline works, even before it's very accurate.

Concepts You Need Today

- **Neural network** — think of it as a very large chain of simple math steps, arranged in layers, that turns "pixel values in" into "a prediction out." No single step is smart; the *pattern* that emerges from millions of trained numbers is what does the work.
- **Transfer learning** — instead of teaching a neural network to recognize edges, shapes, and colors completely from scratch (which needs millions of photos and days of compute), we start from a network **already trained on 1.4 million general photos** (called **ImageNet**) that already "knows" how to see basic visual structure, and we only teach it the new, specific task: diabetic retinopathy signs. This is why we can get a working model from ~3,700 photos in one day instead of needing millions of photos and weeks of training.
- **Backbone** — the pretrained "already knows how to see" part of the network we borrow (we use one called **MobileNetV2** — "Mobile" because it's specifically designed to be small and fast enough for phones, which matters a lot in Day 4).
- **Training / validation / test split** — we never let the model "study" and "take the exam" on the same photos. We hold back some photos it never sees during training (validation, used to tune along the way) and some it *only* sees once, at the very end (test, used to report honest, final numbers).
- **Class imbalance** — our dataset has more "No urgent referral" photos than "Refer" photos. Without correcting for this, the model could get high overall accuracy just by lazily predicting "No urgent referral" every time — which would be a completely useless (and dangerous) screening tool. We correct for this with **class weights**, which tell the model "pay extra attention to the minority class."
- **Sensitivity (recall)** — of all the photos that *actually* needed a referral, what fraction did the model correctly catch? For a screening tool, this is the single most important number: missing a real case (a **false negative**) is a far worse mistake than an unnecessary referral (a **false positive**).

Step-by-Step: First Training Run

Continue in the same notebook (Colab or PAI-DSW) from Day 1 — your `train_binary.csv` and `train_images` folder should still be there (reload from Google Drive with the Day 1 Step 5 commands if you

started a fresh Colab session).

Step 1 — Split the data and set up image loading

PYTHON

```
import pandas as pd
import tensorflow as tf

from tensorflow.keras.applications.mobilenet_v2 import preprocess_input
from tensorflow.keras.preprocessing.image import ImageDataGenerator
from sklearn.model_selection import train_test_split

df = pd.read_csv("train_binary.csv")
df["filename"] = df["id_code"] + ".png"

# 70% train, 15% validation, 15% test – split by the label so each set has a
# similar proportion of "Refer" cases (this is "stratified" splitting).
train_df, temp_df = train_test_split(df, test_size=0.3, stratify=df["refer_label"], random_state=42)
val_df, test_df = train_test_split(temp_df, test_size=0.5, stratify=temp_df["refer_label"], random_state=42)
print(f"Train: {len(train_df)} Validation: {len(val_df)} Test: {len(test_df)}")

IMG_SIZE = (224, 224)
BATCH_SIZE = 32

# preprocess_input scales pixels to the exact range MobileNetV2 expects: -1 to 1.
# We deliberately use ONLY resize + this scaling – no extra cropping or contrast
# correction – because we will need to reproduce these exact same steps inside
# the Flutter app in Day 5, in a completely different programming language.
# Keeping preprocessing this simple is what makes that "parity" reliably achievable
# for a first-time builder in one week.
datagen = ImageDataGenerator(preprocessing_function=preprocess_input)

def make_generator(dataframe, shuffle):
    return datagen.flow_from_dataframe(
        dataframe, directory="train_images", x_col="filename", y_col="refer_label",
        target_size=IMG_SIZE, class_mode="raw", batch_size=BATCH_SIZE, shuffle=shuffle
    )

train_gen = make_generator(train_df, shuffle=True)
val_gen = make_generator(val_df, shuffle=False)
test_gen = make_generator(test_df, shuffle=False)
```

WARNING

This preprocessing decision is a deliberate change from a more complex "circular crop + contrast correction" approach some published diabetic retinopathy papers use. We simplified it on purpose: matching complex image processing exactly between Python and Dart is a common, hard-to-debug source of bugs (the model behaves differently at training time vs. real-world use), and getting that wrong silently produces bad predictions. Plain resize + standard normalization is easy to replicate exactly in both languages, which matters more for a reliable one-week beginner build than squeezing out a small amount of extra accuracy.

Step 2 — Build the model using transfer learning

PYTHON

```
from tensorflow.keras import layers, models

base_model = tf.keras.applications.MobileNetV2(
    input_shape=(224, 224, 3), include_top=False, weights="imagenet"
)
base_model.trainable = False # freeze the pretrained part for this first run

model = models.Sequential([
    base_model,
    layers.GlobalAveragePooling2D(), # turns the backbone's output into one flat list of
    # numbers
    layers.Dropout(0.3), # randomly ignores 30% of values during training to
    # reduce overfitting
    layers.Dense(1, activation="sigmoid") # outputs one number between 0 and 1: "how like
    # ly is Refer"
])

model.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=1e-3),
    loss="binary_crossentropy",
    metrics=["accuracy", tf.keras.metrics.AUC(name="auc"),
             tf.keras.metrics.Recall(name="recall"), tf.keras.metrics.Precision(name="prec
    ision")]
)
model.summary()
```

What's happening: `include_top=False` means "give me MobileNetV2 minus its original 1000-category output layer" — we're keeping only its "how to see" part. `base_model.trainable = False` **freezes** those pretrained numbers so today's training only adjusts our new, small final layer. `Dense(1, activation="sigmoid")` is that new final layer — a single neuron whose output we interpret as a probability.

Step 3 — Correct for class imbalance, then train

PYTHON

```
from sklearn.utils.class_weight import compute_class_weight
import numpy as np
```

```

weights = compute_class_weight(class_weight="balanced", classes=np.array([0, 1]), y=train_
df["refer_label"])
class_weight = {0: weights[0], 1: weights[1]}
print("Class weights:", class_weight)

history = model.fit(
    train_gen,
    validation_data=val_gen,
    epochs=8,
    class_weight=class_weight
)

```

This will take a few minutes (faster with a GPU runtime — double check Runtime → Change runtime type still shows GPU if this feels slow). Watch the printed `loss`, `accuracy`, `val_accuracy` etc. after each **epoch** (one full pass through all training photos) — you should see `val_auc` generally trend upward.

Step 4 — Evaluate honestly on the held-out test set

```

PYTHON
from sklearn.metrics import confusion_matrix, classification_report, roc_auc_score

test_gen.reset()
probs = model.predict(test_gen).flatten()
preds = (probs >= 0.5).astype(int)
true = test_df["refer_label"].values

print("Confusion matrix (rows = actual, columns = predicted):")
print(confusion_matrix(true, preds))
print(classification_report(true, preds, target_names=["No urgent referral", "Refer"]))
print("ROC-AUC:", roc_auc_score(true, probs))

```

Step 5 — Save this first model version

```

PYTHON
model.save("basirah_v1.keras")
# Colab: also copy it somewhere that survives a session reset
!cp basirah_v1.keras "/content/drive/MyDrive/basirah/models/"

```

CHECKPOINT

Checkpoint 3 — The AI training pipeline works.

You should see a confusion matrix (four numbers in a 2x2 grid) and a classification report with a `recall` value for the "Refer" row. **The exact numbers don't matter yet — a working, honest pipeline that prints real numbers is the milestone today, not a polished accuracy score.** If `recall` for "Refer" is very low (near 0), don't panic — Day 3 improves this by fine-tuning. If you instead get an error, check the table below.

Troubleshooting Day 2

	Fix	
Invalidated image filenames	Your <code>directory="train_images"</code> path is wrong relative to where your notebook is running, or <code>filename</code> values don't end in <code>.png</code> . Run <code>!ls train_images</code>	head` to confirm the actual folder name and file
Extremely slow (many minutes per epoch)	Your notebook isn't actually using a GPU. Recheck Runtime → Change runtime type → GPU, then re-run all cells from the top (switching runtime type restarts the whole session).	
Out of memory / out of memory	Lower <code>BATCH_SIZE</code> from 32 to 16 and re-run.	
Loss is 0.00 after training	Confirm <code>class_weight</code> printed two clearly different numbers (not both ~1.0) — if <code>compute_class_weight</code> was run on the wrong column, imbalance correction silently does nothing.	
Times out mid-training	Free-tier Colab disconnects idle sessions. Re-run Day 1 Step 5 to reload your data from Drive, then re-run today's cells — this is exactly the friction Alibaba Cloud PAI-DSW avoids, so consider switching if it keeps happening.	

Save your progress to GitHub the same way as Day 1 (`git add .`, `git commit -m "..."`, `git push`) — copy your final notebook code into `ml/src/train.py` first so it's version-controlled, not just sitting in a cloud notebook.

DAY 3 — Improving the Model and Exporting It for the Phone

Day Objective

By the end of today, you will have a **fine-tuned, quantized .tflite file** — the actual AI model file that will be bundled inside the Flutter app — and you'll have proven it still gives sane predictions after conversion.

Concepts You Need Today

- **Fine-tuning** — after training just the new final layer (Day 2), we now "unfreeze" some of the pretrained backbone's upper layers too, and continue training everything together at a much smaller learning rate. This lets the model adjust its general "vision" slightly toward the specific visual patterns of retinal photos, usually improving accuracy further.
- **Quantization** — shrinking a model file by storing its millions of internal numbers with less precision (for example, using 16-bit numbers instead of 32-bit). This can shrink a model by roughly 4x with only a tiny, usually unnoticeable, accuracy cost — essential for keeping the app small and fast on low-end phones.
- **.tflite file** — the actual exported model file format that phones can run. Everything before this step (Keras, `.keras` files) only exists on your training computer; **the phone only ever sees the .tflite file**.
- **Operating threshold** — the model outputs a probability between 0 and 1. We decide the exact cutoff point that becomes "Refer" vs. "No urgent referral." The default is 0.5, but for a screening tool we often lower it slightly, deliberately trading a few more false alarms for catching more real cases — because missing a real case is worse.

Step-by-Step: Fine-Tune, Quantize, and Export

Step 1 — Unfreeze the top of the backbone and fine-tune

PYTHON

```

base_model.trainable = True

# Keep the earlier (more general) layers frozen; only fine-tune the later,
# more specialized layers. MobileNetV2 has ~155 layers; unfreezing the last ~30
# is a reasonable, well-established starting point.
for layer in base_model.layers[:-30]:
    layer.trainable = False

model.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=1e-5), # much smaller than Day 2's 1
    e-3
    loss="binary_crossentropy",
    metrics=["accuracy", tf.keras.metrics.AUC(name="auc"),
            tf.keras.metrics.Recall(name="recall"), tf.keras.metrics.Precision(name="prec
ision")]
)

fine_tune_history = model.fit(
    train_gen,
    validation_data=val_gen,
    epochs=6,
    class_weight=class_weight
)

```

Re-run Day 2 Step 4's evaluation code afterward and compare the new [recall/ROC-AUC](#) numbers to before. If they got clearly *worse*, that's a sign of overfitting — skip fine-tuning and ship the Day 2 model instead; a slightly-less-accurate honest model beats an overfit one. (See §14 of the earlier blueprint's risk table — this exact fallback was planned for.)

Step 2 — Pick your operating threshold**PYTHON**

```

import numpy as np

probs = model.predict(test_gen).flatten()
true = test_df["refer_label"].values

for threshold in [0.3, 0.4, 0.5]:
    preds = (probs >= threshold).astype(int)
    tp = ((preds == 1) & (true == 1)).sum()
    fn = ((preds == 0) & (true == 1)).sum()
    fp = ((preds == 1) & (true == 0)).sum()
    tn = ((preds == 0) & (true == 0)).sum()
    sensitivity = tp / (tp + fn) if (tp + fn) else 0
    specificity = tn / (tn + fp) if (tn + fp) else 0

```

```
print(f"threshold={threshold}: sensitivity={sensitivity:.2f} specificity={specificity:.2f}")
```

Pick the threshold that gives the highest sensitivity you're comfortable with, without specificity collapsing to near 0 (which would mean the app refers almost everyone, which isn't useful either). Write this number down — you'll hard-code it into the Flutter app on Day 4.

Step 3 — Convert to TFLite with quantization

PYTHON

```
converter = tf.lite.TFLiteConverter.from_keras_model(model)
converter.optimizations = [tf.lite.Optimize.DEFAULT]
converter.target_spec.supported_types = [tf.float16] # FP16 quantization

tflite_model = converter.convert()

with open("basirah_model.tflite", "wb") as f:
    f.write(tflite_model)

import os
print(f"Model size: {os.path.getsize('basirah_model.tflite') / 1024 / 1024:.2f} MB")
```

You should see a file size in the low single-digit megabytes. Download it (in Colab: click the folder icon on the left sidebar, right-click `basirah_model.tflite`, **Download**; in PAI-DSW: use the file browser's download button) — you'll place it inside the Flutter app on Day 4.

Step 4 — Sanity-check the exported file actually still works

Always verify a converted model before trusting it — conversion bugs are a real, if uncommon, failure mode.

PYTHON

```
interpreter = tf.lite.Interpreter(model_path="basirah_model.tflite")
interpreter.allocate_tensors()
input_details = interpreter.get_input_details()
output_details = interpreter.get_output_details()
print("Expected input shape:", input_details[0]['shape'])

# Run one real test image through it
sample_probs = model.predict(test_gen, steps=1)[:1]
sample_batch = next(iter(test_gen))[0][:1]

interpreter.set_tensor(input_details[0]['index'], sample_batch.astype(input_details[0]['dtype']))
interpreter.invoke()
tflite_output = interpreter.get_tensor(output_details[0]['index'])

print("Original Keras model output:", sample_probs)
print("Converted TFLite model output:", tflite_output)
```

CHECKPOINT**Checkpoint 4 & 5 — You have a working, exported AI model.**

The printed `Expected input shape` should read `[1, 224, 224, 3]`. The two printed output numbers (Keras vs. TFLite) should be very close to each other (within about 0.01) — that confirms quantization didn't break the model. If they're wildly different, re-run Step 3 without the `supported_types = [tf.float16]` line (full-precision conversion) as a fallback, and note this as a known limitation in `docs/EVALUATION_RESULTS.md`.

Troubleshooting Day 3

Problem	Fix
Fine-tuning makes validation numbers worse, not better	Skip fine-tuning; export the Day 2 (head-only) model instead. This is a planned, acceptable fallback, not a failure.
<code>converter.convert()</code> throws an error about unsupported ops	Rare with MobileNetV2, but if it happens, remove the <code>target_spec.supported_types</code> line and convert without FP16 quantization first, to confirm the base conversion works, then re-add quantization.
Downloaded <code>.tflite</code> file won't open / seems corrupted	Re-download — browser downloads of binary files occasionally truncate. Confirm the file size printed in Step 3 matches the downloaded file's size on your computer.

End today by pushing to GitHub as usual. Place the final `basirah_model.tflite` into `ml/models/` in your repo (this small file is fine to commit — it's your own trained output, not a redistribution of the raw dataset).

DAY 4 — Building the Flutter App and Wiring Up the AI Model**Day Objective**

By the end of today, you have a real, launchable phone app that can pick a photo and run it through your trained AI model — even if the result screen is still ugly. Today is about proving the hardest technical connection (Dart ↔ TFLite) works at all, before Day 5 makes it look and feel right.

Concepts You Need Today

- **Widget** — in Flutter, literally everything on screen (a button, a bit of text, an image, even an entire screen) is a "widget." You build a screen by nesting widgets inside each other, like nesting boxes.
- **StatefulWidget / setState** — a widget that can change what it displays over time (e.g., "show a loading spinner, then show a result"). Calling `setState(() { ... })` is how you tell Flutter "some value changed, please redraw the screen." We're deliberately using only this simple pattern for Basirah instead of more advanced state-management libraries — it's the right amount of complexity for a linear, single-screen-flow MVP.
- **pubspec.yaml** — your Flutter project's master settings file: it lists every external package (pre-written code library) your app depends on, and which files should be bundled inside the app as "assets" (like our `.tflite` model file and, later, our Urdu font).
- **Package / dependency** — a chunk of code someone else wrote and published, that you can pull into your own project instead of writing that functionality yourself. `tflite_flutter` (runs AI models) and `image_picker` (opens the camera/gallery) are both packages.

- **Emulator vs. physical device** — an emulator is a virtual Android phone that runs *inside* your computer. A physical device is your actual phone, connected by USB. **We recommend a real phone** for this project: it's faster to test on, and lets you use the real camera.

Setup — Install Flutter and Android Tools

1. Install the Flutter SDK

- Download the Flutter SDK zip from docs.flutter.dev/get-started/install/windows.
- Extract it to `C:\src\flutter` (avoid paths with spaces, like `Program Files`, and avoid `C:\Program Files`).
- Add Flutter to your PATH: Windows search → "Edit the system environment variables" → **Environment Variables** → under "User variables," select **Path** → **Edit** → **New** → paste `C:\src\flutter\bin` → OK everything.
- Open a **new** PowerShell window (important — old windows won't see the PATH update) and run:

```
POWERSHELL
flutter --version
```

You should see version info. If not, double-check the PATH entry and reopen PowerShell again.

2. Install Android Studio

Download from developer.android.com/studio and run the installer with default settings. On first launch, its Setup Wizard downloads the **Android SDK** — let it finish; this can take a while on slower internet.

3. Install the VS Code Flutter extension

In VS Code: click the Extensions icon (left sidebar) → search "Flutter" → install the official one by "Dart Code" (it automatically installs the Dart extension too).

4. Run the doctor check

```
POWERSHELL
flutter doctor
```

This scans your computer and tells you what's missing with **X** marks. The most common one for beginners:

```
POWERSHELL
flutter doctor --android-licenses
```

Type **y** and press Enter for every license prompt this shows. Re-run `flutter doctor` afterward — aim for checkmarks on "Flutter," "Android toolchain," and either "Android Studio" or "VS Code." (You don't strictly need every single line green — a warning about Chrome or Visual Studio for web/desktop development doesn't matter for us; we're Android-only.)

5. Connect your Android phone

- On your phone: **Settings** → **About phone**, tap **Build number** seven times (this unlocks Developer Options — a genuinely real, well-known Android feature, not a trick).
- **Settings** → **System** → **Developer options** → **USB debugging** → **On**.
- Plug your phone into your computer with a USB cable. On the phone, a popup asks to trust this computer — tap **Allow**.

→ Confirm your computer sees it:

```
POWERSHELL
flutter devices
```

You should see your phone listed by model name.

TIP

No phone available yet? Skip to the emulator instead: in Android Studio, **More Actions** → **Virtual Device Manager** → **Create Device**, pick any modern phone profile, and start it. It's slower but works identically for these steps.

Step-by-Step: Create the App and Wire Up the Model

Step 1 — Create the Flutter project

In PowerShell, from your `D:\basirah` folder:

```
POWERSHELL
flutter create app
cd app
flutter pub add tflite_flutter image_picker
```

What just happened: `flutter create app` generates a complete, working (if empty) Flutter project inside your existing `app` folder. `flutter pub add` fetches the latest version of each package, adds it to `pubspec.yaml` automatically, and downloads it — no manual version-number editing needed.

Step 2 — Add your trained model as a bundled asset

Create the folder and copy your file in:

```
POWERSHELL
mkdir assets\models
copy ..\ml\models\basirah_model.tflite assets\models\
```

Now open `app/pubspec.yaml` in VS Code. Find the `flutter:` section near the bottom (it already exists, added by `flutter create`) and add the `assets:` lines so it looks like this:

```
YAML
flutter:
  uses-material-design: true
  assets:
    - assets/models/basirah_model.tflite
```

WARNING

YAML files are picky about spacing — use exactly 2 spaces per indent level, never tabs. If Flutter later complains it can't find your asset, the most common cause is misaligned spacing here.

Step 3 — Write the preprocessing code

Create `app/lib/pipeline/preprocessing.dart`:

DART

```
import 'package:image/image.dart' as img;

/// Resizes any input photo to exactly the 224x224 pixel size our AI model
/// expects. This must produce the SAME result as the Python resizing step
/// from Day 2 – that's what "parity" between training and the app means.
img.Image resizeForModel(img.Image original) {
  return img.copyResize(original, width: 224, height: 224);
}
```

Step 4 — Write the inference service

Create `app/lib/pipeline/inference_service.dart`:

DART

```
import 'package:image/image.dart' as img;
import 'package:tflite_flutter/tflite_flutter.dart';

/// Loads the bundled AI model once, and runs it on a preprocessed image.
class InferenceService {
  Interpreter? _interpreter;

  Future<void> loadModel() async {
    _interpreter = await Interpreter.fromAsset(
      'assets/models/basirah_model.tflite',
    );
  }

  /// Takes a 224x224 image and returns a single probability (0.0 to 1.0)
  /// that the image shows signs needing referral to an eye-care professional.
  double runInference(img.Image resizedImage) {
    if (_interpreter == null) {
      throw StateError('Call loadModel() before runInference().');
    }

    // Build the 4D input tensor shape the model expects: [1, 224, 224, 3]
    // Each pixel value is scaled to the -1.0 to 1.0 range, exactly matching
    // the Python training pipeline's preprocess_input step.
    final input = [
      List.generate(resizedImage.height, (y) {
        return List.generate(resizedImage.width, (x) {
          final pixel = resizedImage.getPixel(x, y);
          return [
            (pixel.r / 127.5) - 1.0,

```

```

        (pixel.g / 127.5) - 1.0,
        (pixel.b / 127.5) - 1.0,
      ];
    });
  }),
];

// The model outputs one number, shape [1, 1].
final output = List.generate(1, (_) => List.filled(1, 0.0));

_interpreter!.run(input, output);

return output[0][0];
}
}

```

Step 5 — Write a minimal home screen that picks an image and runs it

Create `app/lib/screens/home_screen.dart`:

```

DART
import 'dart:io';
import 'package:flutter/material.dart';
import 'package:image/image.dart' as img;
import 'package:image_picker/image_picker.dart';
import '../pipeline/inference_service.dart';
import '../pipeline/preprocessing.dart';

class HomeScreen extends StatefulWidget {
  const HomeScreen({super.key});

  @override
  State<HomeScreen> createState() => _HomeScreenState();
}

class _HomeScreenState extends State<HomeScreen> {
  final InferenceService _inferenceService = InferenceService();
  final ImagePicker _picker = ImagePicker();

  File? _selectedImage;
  String _statusText = 'Loading AI model...';
  bool _isBusy = false;

  @override
  void initState() {
    super.initState();
    _loadModel();
  }
}

```

```

}

Future<void> _loadModel() async {
  await _inferenceService.loadModel();
  setState(() => _statusText = 'Model loaded. Pick a photo to analyze.');
```

```

}

Future<void> _pickAndAnalyze() async {
  final picked = await _picker.pickImage(source: ImageSource.gallery);
  if (picked == null) return;

  setState(() {
    _selectedImage = File(picked.path);
    _isBusy = true;
    _statusText = 'Analyzing...';
  });

  final bytes = await _selectedImage!.readAsBytes();
  final decoded = img.decodeImage(bytes!);
  final resized = resizeForModel(decoded);
  final probability = _inferenceService.runInference(resized);

  setState(() {
    _isBusy = false;
    _statusText = 'Raw model output: ${probability.toStringAsFixed(3)}';
  });
}

@override
Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(title: const Text('Basirah (Day 4 test screen)'),
    body: Center(
      child: Column(
        mainAxisAlignment: MainAxisAlignment.center,
        children: [
          if (_selectedImage != null)
            Image.file(_selectedImage!, height: 200),
          const SizedBox(height: 16),
          Text(_statusText),
          const SizedBox(height: 16),
          ElevatedButton(
            onPressed: _isBusy ? null : _pickAndAnalyze,
            child: const Text('Pick Photo & Analyze'),
          ),
        ],
      ),
    ),
  );
}

```

```
    ),  
    );  
  }  
}
```

Step 6 — Wire it up in `main.dart`

Open `app/lib/main.dart`, delete everything in it, and paste:

```
DART  
import 'package:flutter/material.dart';  
import 'screens/home_screen.dart';  
  
void main() {  
  runApp(const BasirahApp());  
}  
  
class BasirahApp extends StatelessWidget {  
  const BasirahApp({super.key});  
  
  @override  
  Widget build(BuildContext context) {  
    return MaterialApp(  
      title: 'Basirah',  
      theme: ThemeData(primarySwatch: Colors.teal, useMaterial3: true),  
      home: const HomeScreen(),  
    );  
  }  
}
```

Step 7 — Run it

With your phone connected (or emulator running):

```
POWERSHELL  
flutter run
```

The first run compiles the whole app and can take a few minutes — later runs are much faster. Once it launches on your phone, tap **"Pick Photo & Analyze,"** choose one of the APTOS sample photos you can transfer to your phone's gallery beforehand (copy a few `.png` files from `ml/data/train_images` onto your phone via USB, WhatsApp-to-self, or Google Drive), and watch the text update.

CHECKPOINT

Checkpoint 6 — The basic app launches. You see "Basirah (Day 4 test screen)" on your phone with a working button.

Checkpoint 7 — On-device inference works. After picking a photo, within a second or two the text changes to something like `Raw model output: 0.734`. **Any number between 0.0 and 1.0 printing without a crash means the entire AI pipeline — Python model, TFLite export, and Dart integration — is correctly connected end to end.** This is the single riskiest technical connection in the whole project, and you've just proven it works.

Troubleshooting Day 4

Problem	Why it happens	Fix
<code>flutter: command not found</code> (after fresh install)	PATH not picked up by current terminal	Open a brand new PowerShell window (or restart VS Code entirely)
<code>flutter doctor</code> shows <code>x</code> for Android toolchain, licenses not accepted	You skipped or half-ran the licenses command	Run <code>flutter doctor --android-licenses</code> again, type <code>y</code> for every single prompt, even ones that look repeated
Unable to locate asset entry in <code>pubspec.yaml</code>	Asset path typo, or wrong indentation	Re-check Step 2's exact YAML spacing; confirm the file really exists at <code>app/assets/models/basirah_model.tflite</code>
App builds but crashes immediately on model load	The <code>.tflite</code> file didn't actually get copied into <code>assets/models/</code> before running, or is 0 bytes	Re-run Step 2's <code>copy</code> command; confirm file size with <code>dir assets\models</code>
<code>Interpreter.fromAsset</code> throws about tensor shape mismatch	Your model's real input/output shape doesn't match what the code assumes	Re-check Day 3's printed <code>Expected input shape</code> — it must read <code>[1, 224, 224, 3]</code> ; if your Keras model architecture differs, adjust the Dart code's dimensions to match
No devices found by <code>flutter devices</code>	USB debugging not enabled, cable is charge-only, or driver missing	Recheck Developer Options → USB Debugging is ON; try a different USB cable/port; on the phone, look for and accept the "Allow USB debugging?" popup (it's easy to miss)
Gradle build fails with a long red error the first time	Very common on a first-ever Flutter Android build — usually a one-time SDK component download issue	Re-run <code>flutter run</code> a second time; if it repeats, run <code>flutter doctor -v</code> and address whatever specific <code>x</code> item it flags

Push to GitHub as usual at the end of the day.

DAY 5 — Quality Checks, Real Results, and Safety Messaging

Day Objective

By the end of today, the raw "0.734" number from Day 4 becomes an actual usable screening result, with a proper results screen and the plain-language safety messaging from our project's medical-safety plan — and bad photos get rejected *before* the model ever sees them.

Concepts You Need Today

- **Heuristic** — a simple, hand-written rule of thumb (not AI) that's "good enough" for a specific job. Our blur and brightness checks are heuristics: cheap, fast, explainable, and don't need any training data of their own.
- **Enum** — a Dart way of defining "this value must be exactly one of these named options" (for example: `poorQuality`, `noUrgentReferral`, `refer`, `uncertain`). It makes your code less error-prone than comparing loose strings or numbers everywhere.
- **Navigation** — moving from one app screen to another. Flutter calls this "pushing a route."

Step-by-Step: Build the Quality Gate

Create `app/lib/pipeline/quality_gate.dart`:

DART

```
import 'package:image/image.dart' as img;

enum QualityCheckResult { pass, tooBlurry, tooDarkOrBright }

/// Cheap, explainable checks that run BEFORE the AI model, so a genuinely
/// unusable photo never produces a false result. These are simple rules,
/// not machine learning – deliberately, to keep this fast, predictable,
/// and easy for a beginner to reason about and debug.
class QualityGate {
  QualityCheckResult check(img.Image original) {
    // Work on a small downsized copy purely for speed – these checks don't
    // need full resolution to be useful.
    final small = img.copyResize(original, width: 150);
    final gray = img.grayscale(small);

    final brightness = _averageBrightness(gray);
    if (brightness < 25 || brightness > 230) {
      return QualityCheckResult.tooDarkOrBright;
    }

    final sharpness = _sharpnessScore(gray);
    if (sharpness < 6.0) {
      return QualityCheckResult.tooBlurry;
    }

    return QualityCheckResult.pass;
  }

  double _averageBrightness(img.Image grayscale) {
    double total = 0;
    for (final pixel in grayscale) {
      total += pixel.r; // grayscale: r, g, and b are all equal
    }
    return total / (grayscale.width * grayscale.height);
  }
}
```



```

/// A simple sharpness proxy: how much brightness changes between
/// neighbouring pixels, on average. Blurry photos have soft, gradual
/// transitions (a low score); sharp photos have crisp edges (a high score).
/// This is NOT a full learned image-quality model – it's an intentionally
/// simple, fast heuristic appropriate for a one-week MVP.
double _sharpnessScore(img.Image grayscale) {
  double totalDiff = 0;
  int count = 0;
  for (int y = 0; y < grayscale.height; y++) {
    for (int x = 0; x < grayscale.width - 1; x++) {
      final p1 = grayscale.getPixel(x, y).r;
      final p2 = grayscale.getPixel(x + 1, y).r;
      totalDiff += (p1 - p2).abs();
      count++;
    }
  }
  return totalDiff / count;
}

```

WARNING

Threshold numbers 25, 230, and 6.0 are starting points, not proven medical constants. Test with a genuinely blurry photo and a genuinely fine photo from your dataset folder, and nudge these numbers if good photos get rejected or bad ones get through. Write down whatever final values you land on in [docs/IMAGE_PIPELINE.md](#).

TIP

We deliberately left out a "does this even look like a fundus photo" check that the original project blueprint considered. Reliable versions of that check need either careful calibration or a small trained model of their own — both real, additional beginner-unfriendly risk. Blur and brightness checks alone still meaningfully protect the demo from the most common bad-input case (an accidental non-fundus photo, or a badly taken one), which is what actually matters this week.

Step-by-Step: Map the AI Output to a Real Result

Create `app/lib/pipeline/result_mapper.dart`:

DART

```

enum ScreeningResult { noUrgentReferral, refer, uncertain }

class ResultMapper {
  // Replace this with the threshold you chose in Day 3, Step 2.
  static const double operatingThreshold = 0.4;
  static const double uncertaintyBand = 0.08;

```

```
static ScreeningResult map(double probability) {
  final distanceFromThreshold = (probability - operatingThreshold).abs();
  if (distanceFromThreshold < uncertaintyBand) {
    return ScreeningResult.uncertain;
  }
  return probability >= operatingThreshold
    ? ScreeningResult.refer
    : ScreeningResult.noUrgentReferral;
}
}
```

Why an "uncertain" band exists: if the model's output lands right on the fence around our cutoff, treating that as a confident answer either way would overstate how sure the model actually is. Routing borderline cases to "please see a professional to be sure" is the safer, more honest choice — this directly implements the medical-safety plan from our original blueprint.

Step-by-Step: Build the Result Screen

Create `app/lib/screens/result_screen.dart`:

```
DART
import 'package:flutter/material.dart';
import '../pipeline/result_mapper.dart';

class ResultScreen extends StatelessWidget {
  final ScreeningResult result;
  final double probability;

  const ResultScreen({
    super.key,
    required this.result,
    required this.probability,
  });

  @override
  Widget build(BuildContext context) {
    final content = _contentFor(result);

    return Scaffold(
      appBar: AppBar(title: const Text('Screening Result')),
      body: Padding(
        padding: const EdgeInsets.all(24),
        child: Column(
          mainAxisAlignment: MainAxisAlignment.center,
          crossAxisAlignment: CrossAxisAlignment.stretch,
          children: [
            Icon(content.icon, size: 72, color: content.color),
            const SizedBox(height: 16),
```

```

        Text(
          content.headline,
          style: Theme.of(context).textTheme.headlineSmall,
          textAlign: TextAlign.center,
        ),
        const SizedBox(height: 16),
        Text(content.message, textAlign: TextAlign.center),
        const SizedBox(height: 24),
        Container(
          padding: const EdgeInsets.all(12),
          decoration: BoxDecoration(
            color: Colors.grey.shade200,
            borderRadius: BorderRadius.circular(8),
          ),
          child: const Text(
            'Basirah is a screening aid, not a diagnosis. It has not '
            'been clinically validated. Always follow up with a '
            'qualified eye-care professional.',
            textAlign: TextAlign.center,
            style: TextStyle(fontSize: 12, color: Colors.black54),
          ),
        ),
        const SizedBox(height: 24),
        ElevatedButton(
          onPressed: () => Navigator.of(context).pop(),
          child: const Text('Screen Another Photo'),
        ),
      ],
    ),
  );
}

_ResultContent _contentFor(ScreeningResult result) {
  switch (result) {
    case ScreeningResult.noUrgentReferral:
      return _ResultContent(
        icon: Icons.check_circle_outline,
        color: Colors.green,
        headline: 'No Urgent Concern Found',
        message:
          'No signs of urgent concern were found in this screening. '
          'This is not a diagnosis. Regular eye check-ups are still '
          'recommended, especially if you have diabetes.',
      );
    case ScreeningResult.refer:
      return _ResultContent(

```

```

        icon: Icons.warning_amber_outlined,
        color: Colors.deepOrange,
        headline: 'Please See an Eye-Care Professional',
        message:
          'This screening found signs that should be checked by an '
          'eye-care professional. Please see an ophthalmologist as '
          'soon as you can. This is not a diagnosis – only a '
          'specialist can confirm what this means.',
      );
    case ScreeningResult.uncertain:
      return _ResultContent(
        icon: Icons.help_outline,
        color: Colors.blueGrey,
        headline: 'Result Not Clear',
        message:
          'This screening could not give a clear result. Please see '
          'an eye-care professional to be sure.',
      );
    }
  }
}

class _ResultContent {
  final IconData icon;
  final Color color;
  final String headline;
  final String message;

  _ResultContent({
    required this.icon,
    required this.color,
    required this.headline,
    required this.message,
  });
}

```

Step-by-Step: Wire Everything Together in `home\_screen.dart`

Open `app/lib/screens/home_screen.dart` and replace the `_pickAndAnalyze` method and imports with this full version:

```

DART
import 'dart:io';
import 'package:flutter/material.dart';
import 'package:image/image.dart' as img;
import 'package:image_picker/image_picker.dart';
import '../pipeline/inference_service.dart';

```

```
import '../pipeline/preprocessing.dart';
import '../pipeline/quality_gate.dart';
import '../pipeline/result_mapper.dart';
import 'result_screen.dart';

class HomeScreen extends StatefulWidget {
  const HomeScreen({super.key});

  @override
  State<HomeScreen> createState() => _HomeScreenState();
}

class _HomeScreenState extends State<HomeScreen> {
  final InferenceService _inferenceService = InferenceService();
  final ImagePicker _picker = ImagePicker();
  final QualityGate _qualityGate = QualityGate();

  File? _selectedImage;
  String _statusText = 'Loading AI model...';
  bool _isBusy = false;

  @override
  void initState() {
    super.initState();
    _loadModel();
  }

  Future<void> _loadModel() async {
    await _inferenceService.loadModel();
    setState(() => _statusText = 'Model loaded. Pick a photo to analyze.');
```

```

    setState(() {
      _isBusy = false;
      _statusText = qualityResult == QualityCheckResult.tooBlurry
        ? "This photo isn't clear enough to check. Please retake it, "
          'holding the camera steady.'
        : "This photo isn't clear enough to check. Please retake it "
          'in better lighting.';
    });
    return; // Stop here – never let a bad photo reach the AI model.
  }

  setState(() => _statusText = 'Analyzing...');

  final resized = resizeForModel(decoded);
  final probability = _inferenceService.runInference(resized);
  final result = ResultMapper.map(probability);

  setState(() => _isBusy = false);

  if (!mounted) return;
  Navigator.of(context).push(
    MaterialPageRoute(
      builder: (_) => ResultScreen(result: result, probability: probability),
    ),
  );
}

@override
Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(title: const Text('Basirah')),
    body: Center(
      child: Padding(
        padding: const EdgeInsets.all(24),
        child: Column(
          mainAxisAlignment: MainAxisAlignment.center,
          children: [
            if (_selectedImage != null)
              Image.file(_selectedImage!, height: 200),
            const SizedBox(height: 16),
            if (_isBusy) const CircularProgressIndicator(),
            const SizedBox(height: 16),
            Text(_statusText, textAlign: TextAlign.center),
            const SizedBox(height: 16),
            ElevatedButton(
              onPressed: _isBusy ? null : _pickAndAnalyze,
              child: const Text('Pick Photo & Analyze'),
            ),
          ],
        ),
      ),
    ),
  );
}

```

```

    ),
    ],
    ),
    ),
    ),
    );
}
}

```

Run it with `flutter run` (or press the hot-reload lightning bolt in VS Code / press `r` in the terminal if it's already running, to apply changes instantly without a full restart).

CHECKPOINT

Checkpoint 8 — The complete workflow works end-to-end. Pick a clearly blurry or very dark photo → you see a "please retake" message and the app never says anything about referral. Pick a normal, clear sample photo → after a brief loading spinner, you land on a full **Result Screen** showing a green "No Urgent Concern" or orange "Please See an Eye-Care Professional" card, with the safety disclaimer visible underneath.

Checkpoint 9 (partial) — Results are displayed correctly. Confirm all three result types are reachable: try several different sample photos until you've seen the green, orange, and (if you can find a borderline one) the grey "uncertain" result at least once each.

Troubleshooting Day 5

Problem	Fix
Every photo gets rejected as "too blurry"	Your <code>sharpness < 6.0</code> threshold is too strict for your test photos' resolution. Print the actual score (<code>print(sharpness)</code> temporarily) on a few known-good photos and set the threshold just below their scores.
Every photo gets rejected as "too dark/bright"	Same idea — print <code>brightness</code> on a few real sample photos and adjust the <code>25/230</code> bounds to fit what you're actually seeing.
App crashes with a null-check error after quality gate passes	Make sure <code>decoded</code> isn't null — <code>img.decodeImage</code> returns null for corrupt/unsupported files; wrap it in a null check and show a friendly "couldn't read this photo" message instead of crashing.
Navigation shows a blank screen	Check <code>ResultScreen</code> 's constructor arguments match exactly what you're passing in <code>Navigator.push</code> — a common typo is forgetting <code>required</code> fields.

Push to GitHub as usual.

DAY 6 — Urdu Translation and Right-to-Left Layout

Day Objective

By the end of today, the app runs fully in both English and Urdu, with a toggle to switch, and Urdu content displays correctly right-to-left in a proper Urdu calligraphic font — the core accessibility requirement of this entire project.

Concepts You Need Today

- **Localization (l10n) / Internationalization (i18n)** — the practice of making an app support multiple languages. "i18n" and "l10n" are just abbreviations (18 and 10 letters skipped, respectively) developers use as shorthand.
- **ARB file** — "Application Resource Bundle," a simple JSON-based file format where you list every piece of text your app shows, keyed by a short name (like `"pickPhotoButton": "Pick Photo & Analyze"`). You write one ARB file per language; the app looks up the right text at runtime based on the selected language.
- **RTL (Right-to-Left)** — Urdu, like Arabic and Hebrew, is read and written right-to-left. A properly localized app doesn't just translate the words — the entire layout mirrors (buttons, alignment, reading order) so it feels natural to an Urdu reader, not just an English layout with different text pasted in.
- **Locale** — a code identifying a language (and sometimes region), like `en` for English or `ur` for Urdu.

Step-by-Step: Set Up Localization

Step 1 — Add the localization packages

POWERSHELL

```
flutter pub add flutter_localizations --sdk=flutter
flutter pub add intl
```

Step 2 — Turn on code generation

Open `app/pubspec.yaml`, find the `flutter:` section, and add `generate: true`:

YAML

```
flutter:
  generate: true
  uses-material-design: true
  assets:
    - assets/models/basirah_model.tflite
```

Step 3 — Create the localization config file

Create a new file `app/l10n.yaml` (in the `app` folder itself, next to `pubspec.yaml` — not inside `lib`):

YAML

```
arb-dir: lib/l10n
template-arb-file: app_en.arb
output-localization-file: app_localizations.dart
output-dir: lib/l10n/generated
synthetic-package: false
```

What this does: tells Flutter's built-in translation tool where to find your ARB files (`lib/l10n`), which one is the "master" list of keys (English), and where to write the generated Dart code it builds from them. Setting

`synthetic-package: false` makes the generated file a normal, visible file in your own project instead of a hidden auto-managed package — this avoids a confusing class of import errors beginners commonly hit with the default settings.

Step 4 — Write the English ARB file

Create `app/lib/l10n/app_en.arb`:

```
JSON
{
  "@@locale": "en",
  "appTitle": "Basirah",
  "pickPhotoButton": "Pick Photo & Analyze",
  "loadingModel": "Loading AI model...",
  "modelReady": "Model loaded. Pick a photo to analyze.",
  "checkingQuality": "Checking photo quality...",
  "analyzing": "Analyzing...",
  "retakeBlurry": "This photo isn't clear enough to check. Please retake it, holding the camera steady.",
  "retakeDark": "This photo isn't clear enough to check. Please retake it in better lighting.",
  "resultTitle": "Screening Result",
  "noReferralHeadline": "No Urgent Concern Found",
  "noReferralMessage": "No signs of urgent concern were found in this screening. This is not a diagnosis. Regular eye check-ups are still recommended, especially if you have diabetes.",
  "referHeadline": "Please See an Eye-Care Professional",
  "referMessage": "This screening found signs that should be checked by an eye-care professional. Please see an ophthalmologist as soon as you can. This is not a diagnosis – only a specialist can confirm what this means.",
  "uncertainHeadline": "Result Not Clear",
  "uncertainMessage": "This screening could not give a clear result. Please see an eye-care professional to be sure.",
  "disclaimer": "Basirah is a screening aid, not a diagnosis. It has not been clinically validated. Always follow up with a qualified eye-care professional.",
  "screenAnotherButton": "Screen Another Photo"
}
```

Step 5 — Write the Urdu ARB file

Create `app/lib/l10n/app_ur.arb`:

```
JSON
{
  "@@locale": "ur",
  "appTitle": "بصیرت",
  "pickPhotoButton": "تصویر منتخب کریں اور جانچیں",
  "loadingModel": "... اے آئی ماڈل لوڈ ہو رہا ہے",
  "modelReady": "ماڈل تیار ہے۔ جانچنے کے لیے تصویر منتخب کریں۔",
  "checkingQuality": "... تصویر کا معیار جانچا جا رہا ہے",
  "analyzing": "... تجزیہ ہو رہا ہے",
}
```

```

"retakeBlurry": "یہ تصویر واضح نہیں ہے۔ براہ کرم کیمرہ ساکت رکھ کر دوبارہ تصویر لیں۔",
"retakeDark": "یہ تصویر واضح نہیں ہے۔ براہ کرم بہتر روشنی میں دوبارہ تصویر لیں۔",
"resultTitle": "اسکریننگ نتیجہ",
"noReferralHeadline": "فوری تشویش کی کوئی علامت نہیں",
"noReferralMessage": "یہ تشخیص نہیں ہے۔ آنکھوں کا باقاعدہ معائنہ اب بھی ضروری ہے، خاص طور پر اگر آپ کو ذیابیطس ہے۔",
"referHeadline": "براہ کرم ماہر امراض چشم سے رجوع کریں",
"referMessage": "جن کا ماہر امراض چشم سے معائنہ ضروری ہے۔ براہ کرم جلد از جلد کسی آنکھوں کے ڈاکٹر سے ملیں۔ یہ تشخیص نہیں ہے۔",
"uncertainHeadline": "واضح نتیجہ نہیں ملا",
"uncertainMessage": "یہ اسکریننگ واضح نتیجہ نہیں دے سکی۔ براہ کرم یقینی بنانے کے لیے آنکھوں کے ماہر سے رجوع کریں۔",
"disclaimer": "ایک اسکریننگ معاون ہے، تشخیص نہیں۔ اس کی طبی توثیق نہیں ہوئی۔ ہمیشہ ایک مستند ماہر امراض چشم سے رجوع کریں۔",
"screenAnotherButton": "دوسری تصویر جانچیں"
}

```

WARNING

These Urdu translations were written to be clear, simple, and medically cautious — but they were not reviewed by a native Urdu-speaking medical-literacy expert. Before showing this to real users (or, ideally, before your final demo), ask a native Urdu speaker to read the four safety messages out loud and confirm they sound natural and unambiguous. This is exactly the kind of review flagged as necessary in our localization plan — plain, everyday wording matters more here than anywhere else in the app.

Step 6 — Generate the Dart code from your ARB files

POWERSHELL

```
flutter pub get
```

Because `generate: true` is set, this reads both ARB files and writes

`lib/l10n/generated/app_localizations.dart` automatically — you never hand-write this file.

Step 7 — Add the Urdu font

- Go to fonts.google.com/noto/specimen/Noto+Nastaliq+Urdu and click **Get font** → **Download all**.
- Unzip the download and find the `.ttf` file inside (it may be named `NotoNastaliqUrdu-Regular.ttf` or similar).
- Copy it into `app/assets/fonts/NotoNastaliqUrdu-Regular.ttf` (create the `fonts` folder).
- Register it in `app/pubspec.yaml`:

YAML

```

fonts:
  - family: NotoNastaliqUrdu
    fonts:
      - asset: assets/fonts/NotoNastaliqUrdu-Regular.ttf

```

(Add this `fonts:` block as a sibling of `assets:` under the same `flutter:` section — same indentation level.)

Step 8 — Wire localization into `main.dart`

Replace the entire contents of `app/lib/main.dart`:

DART

```
import 'package:flutter/material.dart';
import 'package:flutter_localizations/flutter_localizations.dart';
import 'l10n/generated/app_localizations.dart';
import 'screens/home_screen.dart';

void main() {
  runApp(const BasirahApp());
}

class BasirahApp extends StatefulWidget {
  const BasirahApp({super.key});

  @override
  State<BasirahApp> createState() => _BasirahAppState();
}

class _BasirahAppState extends State<BasirahApp> {
  Locale _locale = const Locale('en');

  void _setLocale(Locale newLocale) {
    setState(() => _locale = newLocale);
  }

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Basirah',
      locale: _locale,
      localizationsDelegates: AppLocalizations.localizationsDelegates,
      supportedLocales: AppLocalizations.supportedLocales,
      theme: ThemeData(
        primarySwatch: Colors.teal,
        useMaterial3: true,
        // When Urdu is active, use the bundled Nastaliq font everywhere.
        fontFamily: _locale.languageCode == 'ur' ? 'NotoNastaliqUrdu' : null,
      ),
      home: HomeScreen(currentLocale: _locale, onLocaleChange: _setLocale),
    );
  }
}
```

What's new here: Flutter automatically switches the app's text direction to right-to-left whenever `locale.languageCode` is 'ur' — this happens inside `MaterialApp/Localizations` for you, because Urdu is one of the languages Flutter's framework already knows is RTL. You don't need to manually flip any

layout for the screens we've built, because we used direction-agnostic widgets (`Column`, `Center`, `Text`) throughout Days 4–5.

Step 9 — Update `home_screen.dart` to use translated text and add a language toggle

Open `app/lib/screens/home_screen.dart` and make three changes:

→ Add this import near the top:

```
DART
import '../l10n/generated/app_localizations.dart';
```

→ Change the constructor to accept the current locale and a change callback:

```
DART
class HomeScreen extends StatefulWidget {
  final Locale currentLocale;
  final ValueChanged<Locale> onLocaleChange;

  const HomeScreen({
    super.key,
    required this.currentLocale,
    required this.onLocaleChange,
  });

  @override
  State<HomeScreen> createState() => _HomeScreenState();
}
```

→ Replace every hardcoded English string with the matching localized lookup, and add a language switch button to the `AppBar`. Here is the full updated `build` method and status-text logic:

```
DART
@override
void initState() {
  super.initState();
  _statusText = ''; // set properly once localization context is available
  _loadModel();
}

Future<void> _loadModel() async {
  await _inferenceService.loadModel();
  if (!mounted) return;
  setState(() => _statusText = AppLocalizations.of(context)!.modelReady);
}

Future<void> _pickAndAnalyze() async {
  final l10n = AppLocalizations.of(context)!;
  final picked = await _picker.pickImage(source: ImageSource.gallery);
  if (picked == null) return;
```

```

    setState(() {
      _selectedImage = File(picked.path);
      _isBusy = true;
      _statusText = l10n.checkingQuality;
    });

    final bytes = await _selectedImage!.readAsBytes();
    final decoded = img.decodeImage(bytes!);

    final qualityResult = _qualityGate.check(decoded);
    if (qualityResult != QualityCheckResult.pass) {
      setState(() {
        _isBusy = false;
        _statusText = qualityResult == QualityCheckResult.tooBlurry
          ? l10n.retakeBlurry
          : l10n.retakeDark;
      });
      return;
    }

    setState(() => _statusText = l10n.analyzing);

    final resized = resizeForModel(decoded);
    final probability = _inferenceService.runInference(resized);
    final result = ResultMapper.map(probability);

    setState(() => _isBusy = false);

    if (!mounted) return;
    Navigator.of(context).push(
      MaterialPageRoute(
        builder: (_) => ResultScreen(result: result, probability: probability),
      ),
    );
  }

  @override
  Widget build(BuildContext context) {
    final l10n = AppLocalizations.of(context)!;
    return Scaffold(
      appBar: AppBar(
        title: Text(l10n.appTitle),
        actions: [
          IconButton(
            icon: const Icon(Icons.language),
            tooltip: 'English / اردو',

```

```

        onPressed: () {
          final next = widget.currentLocale.languageCode == 'en'
            ? const Locale('ur')
            : const Locale('en');
          widget.onLocaleChange(next);
        },
      ),
    ],
  ),
  body: Center(
    child: Padding(
      padding: const EdgeInsets.all(24),
      child: Column(
        mainAxisAlignment: MainAxisAlignment.center,
        children: [
          if (_selectedImage != null)
            Image.file(_selectedImage!, height: 200),
          const SizedBox(height: 16),
          if (_isBusy) const CircularProgressIndicator(),
          const SizedBox(height: 16),
          Text(_statusText, textAlign: TextAlign.center),
          const SizedBox(height: 16),
          ElevatedButton(
            onPressed: _isBusy ? null : _pickAndAnalyze,
            child: Text(l10n.pickPhotoButton),
          ),
        ],
      ),
    ),
  ),
);
}

```

Step 10 — Localize `result_screen.dart`

In `app/lib/screens/result_screen.dart`, add the import `import '../l10n/generated/app_localizations.dart'`; and replace the hardcoded English strings in `_contentFor` and the disclaimer `Text` with `AppLocalizations.of(context)!.xxx` lookups, the same pattern as Step 9 (for example, `l10n.noReferralHeadline` instead of `'No Urgent Concern Found'`, and `l10n.disclaimer` instead of the hardcoded disclaimer paragraph). Since `_contentFor` is called from inside `build`, pass `context` (or the already-resolved `l10n`) into it as a parameter.

Step 11 — Update `main.dart`'s call site

Since `main.dart` now passes `currentLocale` and `onLocaleChange` into `HomeScreen`, this is already handled by Step 8's code above — no further change needed. Run:

```

POWERSHELL
flutter run

```

CHECKPOINT

Checkpoint 9 — Urdu/RTL localization works. Tap the language icon in the top-right corner. The entire screen's text switches to Urdu, reading right-to-left, rendered in the Nastaliq calligraphic font — and the layout itself doesn't look "broken" or mirrored strangely. Tap it again to switch back to English. Walk through the full pick-photo → result flow once in each language to confirm every screen (including both possible result screens) shows translated text, not leftover English.

Troubleshooting Day 6

Problem	Fix
Target of URI doesn't exist: 'l10n/generated/app_localizations.dart'	Run <code>flutter pub get</code> again — the file is generated by that command, not created by you; if it still doesn't appear, confirm <code>generate: true</code> is really in <code>pubspec.yaml</code> and <code>l10n.yaml</code> is at the <code>app/</code> root, not inside <code>lib/</code> .
Urdu text shows as boxes (■■■) instead of Urdu letters	The font wasn't registered correctly, or the filename in <code>pubspec.yaml</code> doesn't exactly match the actual file. Re-check Step 7's exact filename and YAML indentation.
Text still reads left-to-right even in Urdu	Confirm the ARB file's <code>"@@locale": "ur"</code> line is present and that you're actually switching to <code>Locale('ur')</code> , not just changing displayed text manually elsewhere.
<code>AppLocalizations.of(context)</code> returns null / crashes	This happens if you call it too early — for example directly inside <code>initState()</code> before the widget tree has fully built. Move any localization lookups inside <code>build()</code> , or into a method safely called <i>after</i> first frame (as Step 9's <code>_loadModel</code> does, guarded by <code>mounted</code>).

Push to GitHub as usual — this is a great point to also update `docs/LOCALIZATION.md` with the note about needing a native-speaker review pass on the Urdu copy.

DAY 7 — Testing, Polish, and Demo Preparation**Day Objective**

By the end of today, you have run through every important test case at least once, built a real installable APK file, installed it on a real phone independent of your development computer, and rehearsed your demo at least once, out loud, with a timer.

Concepts You Need Today

- **Debug build vs. release build** — everything you've run so far with `flutter run` is a **debug build**: slightly larger, slower, includes extra checks to help you catch bugs. A **release build** is the optimized, smaller, faster version you'd actually hand to someone else — this is what you build today for your demo phone and for judges.
- **APK** — "Android Package," the actual installable file format for Android apps (like a `.exe` on Windows, but for Android). `flutter build apk` produces one.

Step-by-Step: Run Through the Full Manual Test Pass

Work through the checklist in **Part C, Section 8 (Testing)** below, one row at a time, on a real phone. Fix anything that breaks before moving on to building the release APK — a broken edge case found today is fixable; one found during the judge demo isn't.

Step-by-Step: Build the Release APK

POWERSHELL

```
cd app
flutter clean
flutter pub get
flutter build apk --release
```

`flutter clean` clears any stale build cache from your many `flutter run` sessions this week — worth doing once before your final build. This will take a few minutes. When it finishes, your installable app file is at:

```
app/build/app/outputs/flutter-apk/app-release.apk
```

Install it on your demo phone, separately from your development connection, to prove it really works standalone:

- Copy `app-release.apk` to your phone (USB transfer, or upload to Google Drive and download it on the phone — email attachments sometimes get blocked by mail apps for `.apk` files).
- On the phone, open the file. If prompted, allow **"Install from this source" / "Install unknown apps"** for whichever app you used to open it (Files, Drive, etc.) — this is a normal Android security prompt for any app installed outside the Play Store, not a sign of a problem.
- Tap **Install**, then open the app and run through the full flow once more.

CHECKPOINT

Checkpoint 10 — Deployment/demo preparation is complete. You have a `.apk` file that installs and runs correctly on a phone with no cable connected to your laptop, no internet connection required, in both languages, producing correct results on both a clear photo and a deliberately bad one.

Step-by-Step: Prepare Your Demo

Read **Part C, Section 10 (Demo Preparation)** below in full, and do a complete timed dry run of your pitch + live demo out loud at least once today — ideally in front of one other person. Note anything that felt confusing or too slow, and fix it now rather than during judging.

Troubleshooting Day 7

Problem	Fix
<code>flutter build apk --release</code> fails with a signing-related error	For a hackathon demo, Flutter's default debug signing config is sufficient — you do not need to set up a real release signing key. If you see a signing error, it usually means a previous partial build left bad state; re-run <code>flutter clean</code> then rebuild.

Problem	Fix
The release APK is much larger than expected	Check that your <code>.tflite</code> model wasn't accidentally duplicated or that you haven't bundled extra unused image assets; also confirm you passed <code>--release</code> , not a debug build, which is larger by design.
App installs but immediately crashes on the demo phone (but worked on your dev phone)	Usually an Android version or architecture mismatch. Confirm the demo phone's Android version is reasonably recent (Android 8+), and if it still fails, fall back to running via cable and <code>flutter run</code> on that same phone as a backup.
You're out of time and something is still broken	Refer to Part C, Section 7 (Hackathon Optimization) below for exactly what to cut first, in priority order, without breaking the core demo.

Final push to GitHub, and update `plan.md`'s checklist to mark Day 7 complete.

PART C — EVERYTHING ELSE YOU NEED

C1. Final Project Structure Explained

```

basirah/
  ■■■ plan.md           ← EDIT constantly. Your daily checklist and living plan.
  ■■■ .gitignore        ← EDIT rarely. Tells Git what to never upload.
  ■■■ docs/
    ■ ■■■ DATASET.md    ← EDIT to record dataset decisions/limitations.
    ■ ■■■ ML_PLAN.md    ← EDIT to record model/training decisions.
    ■ ■■■ EVALUATION_RESULTS.md ← EDIT with your REAL measured numbers from Day 2-3. Never fabricate these.
    ■ ■■■ IMAGE_PIPELINE.md ← EDIT with your final quality-gate threshold values.
    ■ ■■■ LOCALIZATION.md ← EDIT with translation review notes.
    ■ ■■■ MEDICAL_SAFETY.md ← EDIT if you change any user-facing safety wording.
  ■■■ ml/
    ■ ■■■ data/         ← LEAVE ALONE / never commit. Downloaded dataset. Listed in .gitignore.
    ■ ■■■ src/
      ■ ■■■ train.py     ← EDIT this if you re-run or adjust training later.
      ■ ■■■ export_tflite.py ← EDIT only if you change quantization settings.
      ■ ■■■ models/
        ■ ■■■ basirah_model.tflite ← Generated output. Don't hand-edit; regenerate via training.
  ■■■ app/              ← the actual Flutter project
    ■■■ pubspec.yaml    ← EDIT whenever you add a package or asset.
    ■■■ l10n.yaml       ← EDIT rarely (only if you restructure localization).
    ■■■ lib/
      ■ ■■■ main.dart    ← EDIT for app-wide setup (theme, locale, routing).

```

```

■   ■■■ pipeline/
■   ■   ■■■ quality_gate.dart    ← EDIT to tune blur/brightness thresholds.
■   ■   ■■■ preprocessing.dart   ← EDIT ONLY if you also update ml/src training pre
processing to match.
■   ■   ■■■ inference_service.dart ← EDIT rarely — this is the core AI wiring.
■   ■   ■■■ result_mapper.dart    ← EDIT to change the referral threshold.
■   ■■■ screens/
■   ■   ■■■ home_screen.dart      ← EDIT for UI changes to the main screen.
■   ■   ■■■ result_screen.dart    ← EDIT for UI/wording changes to results.
■   ■■■ l10n/
■       ■■■ app_en.arb            ← EDIT to change English text.
■       ■■■ app_ur.arb            ← EDIT to change Urdu text.
■       ■■■ generated/           ← LEAVE ALONE. Auto-generated by `flutter pub get`.
■■■ assets/
■   ■■■ models/basirah_model.tflite ← Copied here from ml/models — don't hand-edit.
■   ■■■ fonts/NotoNastaliqUrdu-Regular.ttf
■■■ android/                     ← LEAVE ALONE unless troubleshooting specifically po
ints you here.
■■■ build/                       ← LEAVE ALONE. Generated build output, already in .g
itignore.

```

Where configuration belongs: all app-level settings (package name, version, dependencies, bundled assets/fonts) live in `app/pubspec.yaml`. Localization settings live in `app/l10n.yaml`. Nothing else in this project needs a separate config file.

Where secrets belong, and what `.env` is: an `.env` file is a plain text file (never committed to Git) holding sensitive values like API keys, in `NAME=value` format, which your code reads at startup instead of having secrets typed directly into source code. **Basirah's finished app doesn't need one** — it has no backend, no API keys, nothing secret at runtime. The *only* real secret in this whole project, your Kaggle token, lives only on your training-cloud machine (Day 1), never inside the `app/` or `ml/` folders that get committed to Git. If a future version of Basirah ever adds something like an optional cloud "second opinion" feature (postponed — see Part C3), that's the point where a real `.env` file and API key would first become necessary.

What should NEVER be uploaded publicly: `ml/data/` (the dataset's license doesn't allow redistribution), `kaggle.json`, any Alibaba Cloud AccessKey files, and any personal information from real test users if you ever collect any (you shouldn't need to for a hackathon demo — use only the public dataset's sample photos).

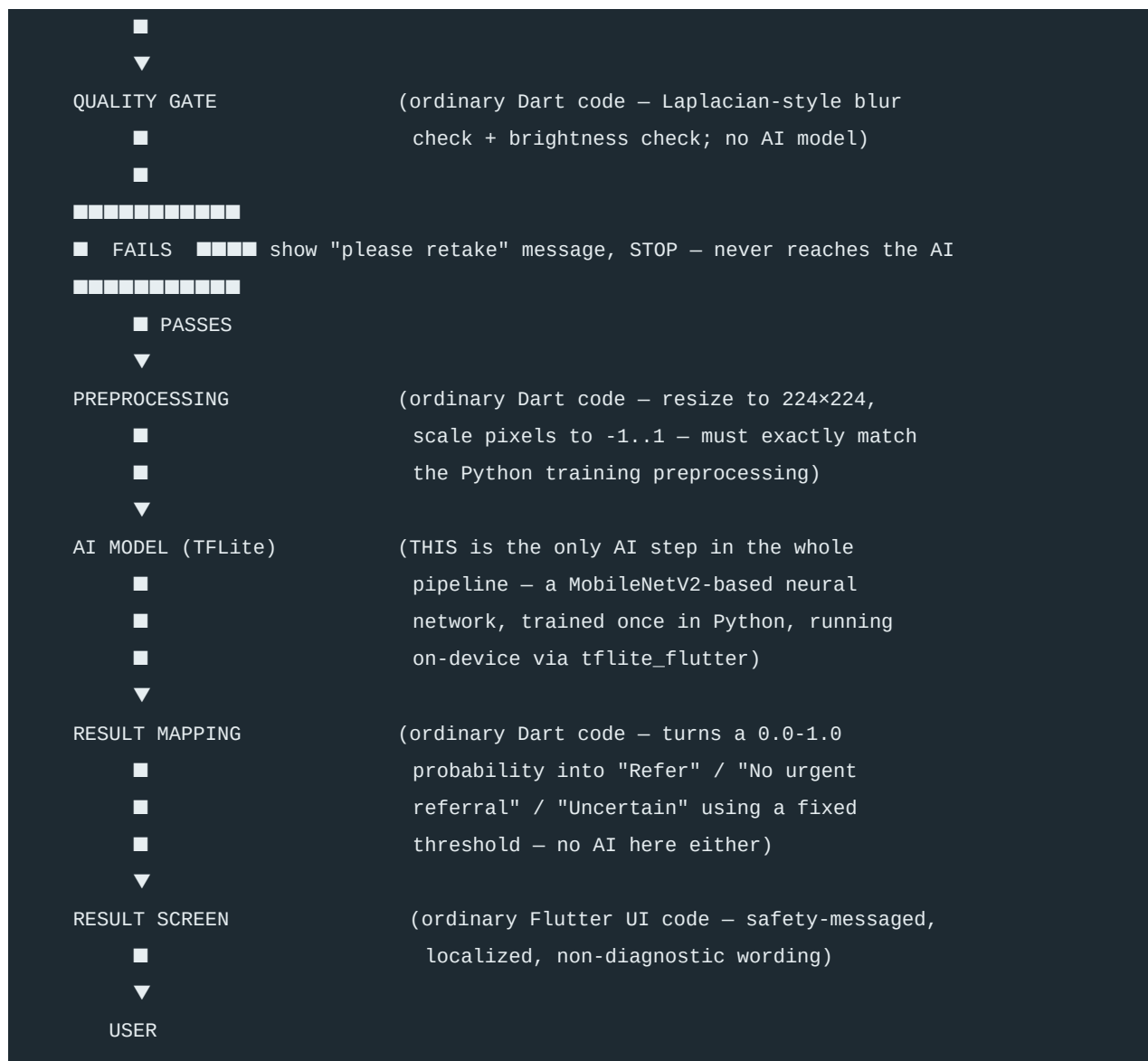
How `.gitignore` works, in one sentence: any file or folder pattern listed in `.gitignore` is invisible to `git add`, `git status`, and `git commit` — Git behaves as if those files don't exist, permanently, until you remove them from the list.

C2. The Complete AI / Image Pipeline, Stage by Stage

```

USER INPUT                                (user picks/takes a fundus photo)
  ■
  ▼
FLUTTER APP UI                            (ordinary app code — no AI here)

```



Notice there is no BACKEND stage and no separate "cloud" step in this diagram — that's not a simplification we're glossing over, it's the actual, deliberate architecture (Part A7). Every stage above except the one labeled "AI MODEL" is completely ordinary application code — no machine learning, no cloud service, nothing "smart" — clearly written Dart logic doing resizing, arithmetic, and if/else branching. **Only the single "AI MODEL" box is where the trained neural network runs**, and everything before and after it exists purely to feed it correctly and present its output responsibly.

WARNING

What the AI model does NOT do: it does not "know" it's looking at an eye, does not understand diabetes, and cannot explain its reasoning. It performs one narrow statistical task — estimating, from learned visual patterns, how likely a photo resembles the "referable DR" examples it was trained on. It has not been clinically validated and must never be presented to a user as a diagnosis. This is why every result screen carries the disclaimer built in Day 5.

C3. Hackathon Optimization — What To Build, In Priority Order

Priority	Features
Must Have (the demo doesn't work without these)	Image import (camera/gallery); quality gate rejecting clearly bad photos; on-device AI inference producing a real result; the three-outcome result screen with safety messaging; English + Urdu with correct RTL; a working release APK installable without your dev laptop connected.
Should Have (build these if the Must-Haves are stable with time to spare)	Polished visual design/branding beyond default Material widgets; an in-app "About / Disclaimer" screen explaining the project and its dataset sources; showing the raw probability or confidence alongside the plain-language result (for judges who want technical detail); basic on-device history of past screenings (local only, never synced).
Nice to Have (only attempt once everything above is rock solid)	Displaying the underlying 5-stage DR grade as secondary detail (per the original blueprint's "both" option); light animations/transitions; app icon and splash screen branding; a short in-app tutorial/walkthrough for first-time users.
Cut First (remove immediately if you're running out of time)	The 5-stage secondary detail display; on-device history; any UI animation polish; the "About" screen (a spoken explanation during the demo covers the same ground); attempting iOS at all; attempting fine-tuning (Day 3) if it's not clearly improving results — ship the Day 2 head-only model instead.

Judging-criteria reminder: a hackathon demo that reliably shows a real, working, on-device AI result in two languages beats a feature-rich app that crashes or needs explaining away. If you must choose between adding a feature and hardening the existing flow against edge cases, harden.

C4. Testing — Complete Beginner-Friendly Checklist

Work through this on a real phone with the release APK (Day 7), not just during development.

Golden path

- Pick a clear, good-quality sample fundus photo showing no DR (APTOS grade 0 or 1) → result is "No Urgent Concern," green.
- Pick a clear sample photo showing moderate-or-worse DR (APTOS grade 2+) → result is "Please See an Eye-Care Professional," orange.
- Do both of the above in **both** English and Urdu.

Bad/invalid input handling

- Pick a deliberately blurry photo → rejected with the retake message; AI never runs.
- Pick a very dark or overexposed photo → rejected with the retake message.
- Pick a completely unrelated photo (e.g., a photo of a wall or a person's face, not an eye) → confirm the app doesn't crash; it will likely still run inference (we deliberately dropped a strict fundus-sanity check — Day 5) and may show a spurious result. **Note this as a known limitation in your demo talking points, not something to hide.**
- Cancel the photo picker without choosing anything → app returns to the idle state without crashing.
- Pick an unusually large photo file (e.g., a 12MP+ camera photo) → confirm processing still completes without freezing or crashing (this can reveal if resizing is happening late/inefficiently).

Failure cases

- Turn on Airplane Mode entirely, then run the full flow → everything still works (this is the core promise of the architecture — actually test it, don't just assume it).
- Force-close the app mid-analysis and reopen it → app returns to a clean starting state, no leftover broken state.

Loading states / UX

- The loading spinner appears during "Checking photo quality..." and "Analyzing..." and disappears correctly afterward, every time.
- The language toggle button is reachable and visibly responds immediately when tapped.

Device/responsive testing

- Test on at least one lower-end/older Android phone if you can borrow one, not only your primary dev phone — this project explicitly targets low-resource devices, so an emulator or top-tier phone alone doesn't prove that goal.
- Rotate the phone to landscape briefly and confirm nothing visually breaks (not a core requirement, but a 5-second check worth doing).

"What if" resilience checks specific to this architecture (no backend, so no API/network failure modes to test):

- Confirm the app never shows any network-related loading state or error (if it does, something was accidentally added that shouldn't be there for this architecture).

Record any checklist item you couldn't fix in time directly in [plan.md](#)'s Known Risks section, worded honestly — judges respect "here's a known limitation and why" far more than a crash they discover themselves.

C5. Deployment — Explained From Zero

What "deployment" normally means: taking a working app off your development computer and making it available in a form other people can actually run — usually by uploading it to a server (for websites) or an app store (for mobile apps).

What deployment means for Basirah, specifically, and why it's simpler than you might expect: because there is no backend server, **there is nothing to host in the cloud.** "Deploying" Basirah means two things:

- Producing the installable `.apk` file (done in Day 7).
- Publishing your source code to GitHub, so judges/reviewers can see and verify how it was built.

We are **not** publishing to the Google Play Store for this hackathon — that process involves developer account fees, review time, and policy compliance steps that are entirely unnecessary for a one-week hackathon MVP demo. A directly-installed `.apk` (called "sideloading") is the standard, expected way to share a hackathon Android demo.

Step-by-step: publish your code to GitHub (the actual "deployment" step for this project)

POWERSHELL

```
git add .
git commit -m "Final Day 7: tested, polished, release APK built"
git push
```

Then, on [github.com](#), open your repository and confirm:

- `plan.md` is visible at the repo root.
- `docs/`, `ml/`, and `app/` folders are all present.
- `ml/data/` does **not** appear (confirming `.gitignore` correctly excluded it).
- No `kaggle.json` file appears anywhere in the repo (search the repo's file list if unsure).

Step-by-step: get a shareable link for your APK, for judges who want to install it themselves

- Upload `app-release.apk` to Google Drive.
- Right-click it → **Share** → **Get link** → set to "Anyone with the link" → **Copy link**.
- Add that link to your GitHub repo's `README.md` and to your final demo slide/notes.

How to verify your "deployment" actually works: on a phone that has never been connected to your development laptop, with Wi-Fi and mobile data both off, download the APK via the shared link on a *different* device that does have internet, transfer it over, and install fresh. If it installs and runs correctly from a completely clean state, your deployment is verified.

Troubleshooting deployment

Problem	Fix
GitHub rejects a push, saying a file is too large	This almost always means <code>ml/data/</code> or a stray large file slipped past <code>.gitignore</code> . Run <code>git status</code> and check nothing under <code>ml/data/</code> is listed as staged; if it is, confirm <code>.gitignore</code> 's spelling exactly matches the folder name.
Google Drive link asks for permission / doesn't just download	Re-check the share setting is "Anyone with the link," not "Restricted."
APK installs but immediately says "App not installed"	Usually means a previous, differently-signed version of the app is already on the phone. Uninstall the old version first, then reinstall.

C6. Demo Preparation — How To Present Basirah To Judges

Before your presentation slot:

- Fully charge your demo phone.
- Have the release APK already installed and working — don't install live in front of judges.
- Preload 3–4 sample fundus photos into the phone's gallery ahead of time: one clearly "no referral" case, one clearly "refer" case, one deliberately blurry photo, one in each language if you're demonstrating localization live.
- Turn on Airplane Mode on the demo phone *before* you start talking, and mention this explicitly — it's your strongest, most concrete proof point.
- Have your GitHub repo and (if built) `docs/EVALUATION_RESULTS.md` open in a browser tab, ready to switch to if asked for technical detail.

Ideal demo flow (aim for 3–4 minutes total):

- **The problem (30 seconds).** State it concretely: diabetic retinopathy is a leading cause of preventable blindness, Pakistan has a severe shortage of eye-care specialists and no widespread screening program, and existing screening tools assume English literacy and internet access neither of which is guaranteed for the people who need this most.
- **The solution, in one sentence.** "Basirah is an on-device, Urdu-first AI screening app that works completely offline."

→ **Live demo, in Airplane Mode.** Pick the "no referral" sample photo → show the quality gate briefly rejecting a deliberately bad photo first (this proves the pipeline is real, not scripted) → then show a good photo producing a clean green result. Switch language to Urdu, live, and show the RTL layout. Then show the "refer" case.

→ **Architecture, briefly (30–45 seconds).** One sentence each: "the AI model is a MobileNetV2-based classifier, fine-tuned on the APTOS 2019 clinical dataset, exported to TensorFlow Lite, and it runs entirely on the phone's own processor — nothing about this photo ever leaves the device." Mention Alibaba Cloud PAI-DSW here if you trained there: "we trained the model using Alibaba Cloud's PAI-DSW GPU environment, provided through this hackathon."

→ **Close with impact, not features.** "This is a screening aid, not a diagnosis — but it's a screening aid that can reach someone in a village with no eye doctor and no internet connection, in their own language."

What's worth mentioning technically (if asked, or if time allows): the on-device/offline architecture and why (privacy + connectivity in Pakistan specifically), the dataset and its honest limitations (single-institution, not Pakistan-sourced — say this proactively, it builds credibility rather than undermining it), the binary "referable DR" framing and why it's more clinically standard and more honest than a flashy 5-stage grade, and your use of Alibaba Cloud's PAI platform for training.

What's not worth spending demo time explaining: Dart/Flutter syntax details, the exact neural network layer structure, Git/GitHub workflow, or anything about `.gitignore`/project setup — none of this is what judges are evaluating.

Likely judge questions and strong answers:

Question	Strong answer
"Is this clinically validated? Would you trust it for real patients?"	"No — and we say that explicitly in the app itself. This is a research-stage screening aid built on a public academic dataset in one week. Its value is as a <i>triage signal</i> to encourage someone to seek a real eye exam, not as a replacement for one."
"What happens if the photo is bad?"	Live-demonstrate the quality gate again if you haven't already, and explain it runs before the AI model ever sees the image.
"Why not just use a cloud API instead of on-device?"	"Two reasons specific to Pakistan: unreliable internet access in the areas that need this most, and because retinal photos are sensitive medical images — keeping them on-device by design means there's nothing to transmit or store insecurely."
"How accurate is it?"	Give your real, measured sensitivity/specificity numbers from <code>docs/EVALUATION_RESULTS.md</code> if you have them, framed honestly: "on our held-out test set from the APTOS dataset, we measured X — but this hasn't been validated on a Pakistani population yet, which we're upfront about." Never state a number you didn't actually measure.
"Why Urdu specifically, and why does that matter technically?"	Explain the accessibility motivation (A2) and that Flutter's localization is a first-class, fully-supported feature, not a hack — you can point at the ARB files and RTL behavior as concrete proof, not just a claim.

C7. Troubleshooting — Master Reference

Problem	Why it happens	How to diagnose	Fix
'command' is not recognized as an internal or external command	The relevant tool (git/flutter/python) isn't on your Windows PATH, or you need a fresh terminal window	Run <code>where git / where flutter / where python</code> — if it prints nothing, it's a PATH problem	Reinstall pointing PATH correctly, or manually add the tool's folder to PATH via Environment Variables, then open a new terminal window
Python version errors / "no module named X"	Multiple Python installs on one machine, or a package installed for the wrong one	Run <code>python --version</code> and <code>pip --version</code> and confirm they reference the same install	Reinstall the missing package with <code>pip install <name></code> ; on cloud notebooks (Colab/PAI-DSW) always use <code>!pip install</code> , not a separate terminal
<code>pip install / flutter pub get</code> fails with a network-looking error	Flaky connection, or a package registry temporarily unreachable	Try again once; check if other downloads on the same network are also slow	Re-run the same command — these are almost always transient
Kaggle download 403 Forbidden	Competition rules not accepted on Kaggle's website yet	Visit the competition page while logged in	Click "I Understand and Accept," then retry the download
<code>kaggle.json</code> errors ("Could not find kaggle.json")	Token file not placed in the exact expected hidden folder	Re-run Day 1 Step 2's four commands	Re-upload and re-copy the file
<code>.gitignore</code> doesn't seem to be working	The file was already tracked by Git <i>before</i> you added it to <code>.gitignore</code>	Run <code>git status</code> — if an ignored file still shows as tracked, that's why	Run <code>git rm --cached <path></code> once to untrack it, then commit again
CORS errors	This is a browser security restriction that blocks a webpage from calling an API on a different domain than it was served from	N/A — Basirah has no backend and no web frontend calling a remote API, so this error class cannot occur in this project. If you ever see the term "CORS" while building Basirah, you've likely pasted in unrelated example code from a different kind of project.	—
Port conflicts ("address already in use")	Two programs trying to use the same network port at once — common in web/backend development	For Basirah, this can only realistically happen if <code>flutter run</code> tries to launch a debug service port that's already busy from a previous crashed session	Fully close any leftover terminal/ <code>flutter run</code> processes, or restart your computer if one seems stuck
"Frontend not connecting to backend"	A very common issue in typical full-stack apps	N/A for Basirah — there is no separate backend for the app to fail to reach. If you see connection-refused-style errors, they're almost certainly about <code>flutter run</code> 's own debug connection to your device, not an app-to-server call	Reconnect the USB cable / restart <code>flutter run</code>

Problem	Why it happens	How to diagnose	Fix
Model gives obviously wrong results for every photo	Preprocessing mismatch between Python and Dart, or wrong operating threshold	Compare a single test image's Python-side prediction (Day 3, Step 4) against the same image run through the app	Re-check <code>preprocessing.dart</code> matches Day 2's Python preprocessing exactly (resize size, normalization formula)
"File not found" errors anywhere	An incorrect relative file path — one of the most common beginner mistakes across every language	Print/log the exact path being used and compare it, character by character, to where the file actually is	Fix the path; prefer running commands from the exact folder the instructions specify (<code>cd app</code> before Flutter commands, for example)
GitHub push rejected / asks to pull first	Your local history and GitHub's history have diverged (rare if you're working solo, but can happen if you edited via GitHub's website too)	Run <code>git status</code>	Run <code>git pull --rebase</code> then <code>git push</code> again

C8. Glossary

- **Frontend** — the part of an app the user directly sees and interacts with. For Basirah, this is the entire Flutter app.
- **Backend** — a separate server-side program an app talks to over the internet. Basirah deliberately has none.
- **API (Application Programming Interface)** — a defined way for one piece of software to request something from another. Think of it as a waiter carrying a request to the kitchen and bringing back the response.
- **Endpoint** — one specific address/URL an API exposes for a particular kind of request (e.g., "get user data"). Not used in Basirah, since there's no API to call.
- **Request / Response** — a request is what one system asks for; a response is what it gets back. Basic building blocks of any API interaction.
- **JSON** — "JavaScript Object Notation," a simple, widely-used text format for structuring data as named key/value pairs, e.g. `{"name": "Basirah"}`. Our ARB translation files are JSON.
- **Database** — a system for storing and querying structured data long-term. Basirah doesn't use one — any local history feature (postponed/optional) would use simple on-device storage instead, not a full database.
- **Model** — in AI, a file containing millions of learned numbers that, together, let a program make a prediction from new input. Distinct from "model" in the general software sense (like a "data model").
- **Inference** — running an already-trained AI model on new input to get a prediction, as opposed to "training," which is the one-time process of teaching it.
- **Machine learning** — a field of computer science where, instead of a programmer writing explicit rules, a program learns patterns automatically from example data.
- **Computer vision** — the subfield of machine learning focused on understanding images/video.
- **Framework** — a large, structured toolkit that shapes how you build an entire application (Flutter is a framework).

- **Library** — a smaller, more focused piece of reusable code you pull into your project for one job (the `image` package is a library).
- **Dependency / Package** — external code your project relies on, usually installed via a package manager (`flutter pub add`, `pip install`).
- **Repository ("repo")** — one project's tracked folder of files plus its entire saved history, as understood by Git.
- **Git** — the version-control program that tracks every saved change to your project's files on your own computer.
- **GitHub** — a website that hosts a copy of Git repositories online, for backup and sharing.
- **Environment variable** — a named piece of configuration data (often a secret) kept outside your source code, typically loaded from a `.env` file.
- **Deployment** — making a finished piece of software available for other people to actually use. For Basirah, this means producing an installable APK and publishing the source code, not hosting a server.
- **Hosting** — running a program (usually a server) continuously so others can reach it online. Not needed for Basirah.
- **Localhost** — a name meaning "this same computer," used when a program is running and reachable only on your own machine, not the internet. Relevant during Flutter development (your emulator/phone briefly talks to your dev machine over a local debug connection) but not part of Basirah's actual architecture.
- **Port** — a numbered "channel" a networked program listens on, allowing multiple programs on one machine to each have their own address.
- **CORS (Cross-Origin Resource Sharing)** — a browser security rule restricting which websites a webpage's code is allowed to fetch data from. Doesn't apply to Basirah (no web frontend, no backend to call).
- **Authentication** — proving who you are to a system (logging in). Basirah has no accounts and no authentication.
- **Cloud computing** — renting computing power (storage, processing, GPUs) from a remote provider over the internet instead of owning the hardware yourself. We use this only temporarily, for AI training (Alibaba Cloud / Colab) — never as part of the finished app.
- **Widget** — in Flutter, any on-screen UI building block.
- **State / StatefulWidget** — data that can change while the app runs, and the specific kind of Flutter widget that's allowed to hold and react to it.
- **Transfer learning** — reusing a neural network already trained on a large general dataset as the starting point for a new, more specific task, instead of training from nothing.
- **Quantization** — reducing the numeric precision used to store a trained model's internal numbers, to shrink its file size and speed it up, usually with minimal accuracy loss.
- **ARB file** — the JSON-based translation file format Flutter's official localization tooling uses.
- **RTL (Right-to-Left)** — describes languages, like Urdu, read and laid out right-to-left; a properly localized app mirrors its whole layout accordingly.